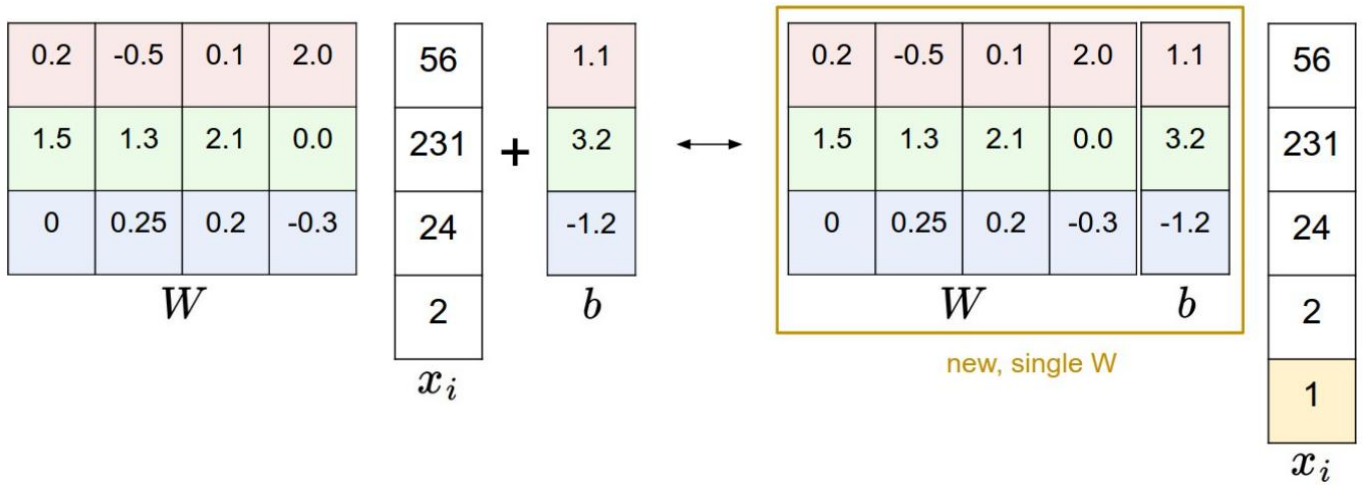


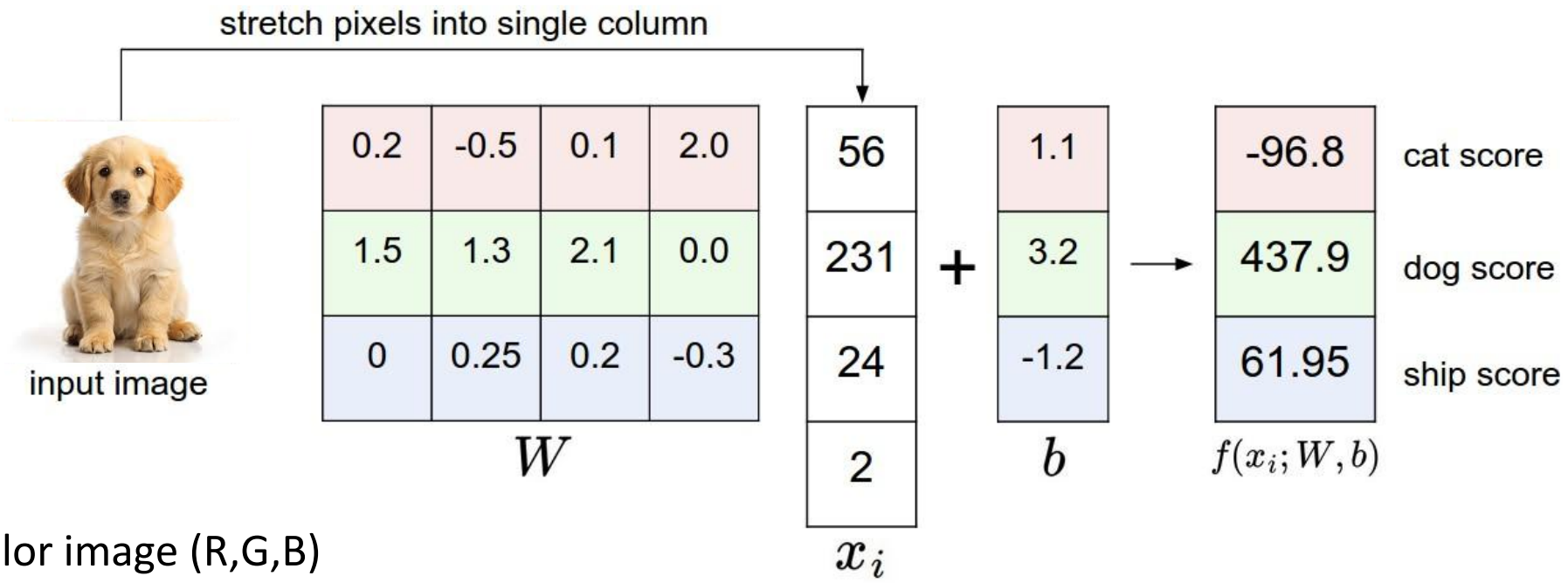
EN3150 Pattern Recognition
Classification
Part 02

M. T. U. Sampath K. Perera,
Department of Electronic and Telecommunication Engineering,
University of Moratuwa.
(sampathk@uom.lk).
Semester 5 – Batch 23.

Linear Classifier

https://cs231n.github.io/linear-classify/





Each row corresponds to a class (e.g., cat, dog, ship), and each column corresponds to a feature. It transforms the input vector into class-specific scores

During training, the model adjusts **W** and **b** to minimize a loss function (e.g., cross-entropy or L2 loss) (maximize the score)

32 Color image (R,G,B)
D=3X 32 X 32=3072

$[K \text{ x } D]$ $[D \text{ x } 1]$ $[K \text{ x } 1]$ $[K \text{ x } 1]$

$$f(x_i, W, b) = Wx_i + b$$

Linear Classifier

Interpretation of linear classifiers as template matching

- Each row of weights W acts as a template (or prototype) for a class.
- The score for each class is determined by comparing the image to each template using the inner product (dot product).
- The linear classifier performs template matching with learned templates



- The horse template appears to feature a two-headed horse because it combines left- and right-facing horses from the dataset.
- The car classifier merges several variations into one template that identifies cars from all angles and colors
- The resulting template is predominantly red, indicating that red cars are more common in the CIFAR-10 dataset.
- The linear classifier is too weak to properly account for different-colored cars
- Neural networks can develop intermediate neurons in hidden layers to detect specific car types (e.g., green car facing left, blue car facing front).

https://cs231n.github.io/linear-classify/

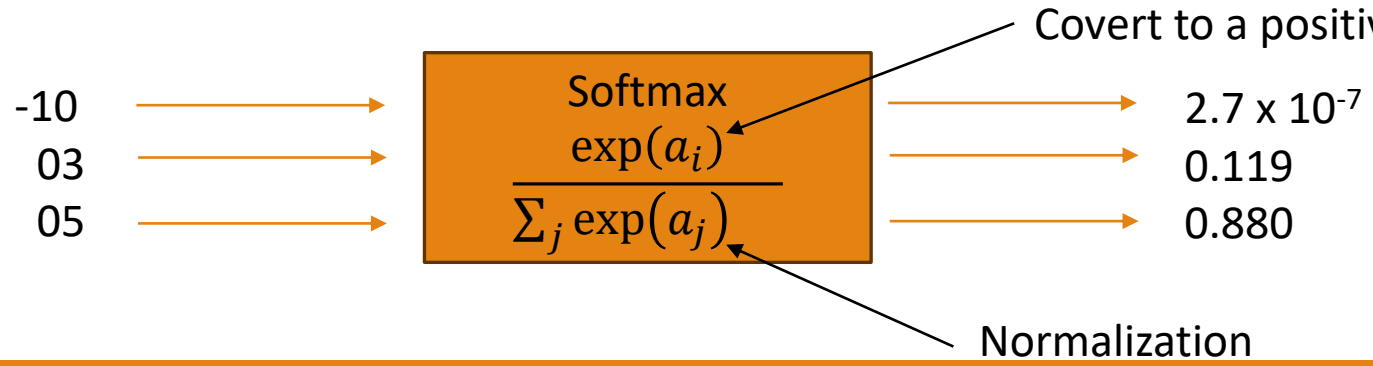
Multinomial logistic regression

- Also known as multiclass logistic regression
- For multiple K classes, there are K linear functions

$$p(y_i = C_k|\boldsymbol{x_i}, \mathbf{W}) = \frac{\exp(w_{0k}+w_{1k}x_{1,i}+\cdots+w_{Dk}x_{D,i})}{\sum_j \exp(w_{0k}+w_{1k}x_{1,i}+\cdots+w_{Dk}x_{D,i})} = \frac{\exp(\boldsymbol{w}_k^T\boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T\boldsymbol{x}_i)}$$

Softmax transformation

- $\mathbf{W}$ is K x (D+1) weight matrix and D is no of features.
- $p(y_i|x_i, \mathbf{W}) = \text{Cat}\,(y_i|\text{softmax}\,(\widetilde{\boldsymbol{W}}\boldsymbol{x}_i + \boldsymbol{b}))$. Here, $\boldsymbol{b} = [w_{01}, \ldots, w_{0K}]^T$ is a K length vector*.



SoftMax

- Preserves the order of the input

-10 < 03 < 05 and 2.7 x 10^{-7} < 0.119 < 0.880

- Values are between 0 and 1
- Values sum up to 1

* $\boldsymbol{b}$ can be added to first column by considering dummy feature equal to 1, $\mathbf{W}= [\boldsymbol{b}\ \widetilde{\boldsymbol{W}}]$.

Multinomial logistic regression

$$\begin{bmatrix} p(y_i = C_1|x_i, \mathbf{W}) \\ p(y_i = C_2|x_i, \mathbf{W}) \\ \vdots \\ p(y_i = CK|x_i, \mathbf{W}) \end{bmatrix} = \begin{bmatrix} \frac{\exp(\boldsymbol{w}_1^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} \\ \frac{\exp(\boldsymbol{w}_2^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} \\ \vdots \\ \frac{\exp(\boldsymbol{w}_K^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} \end{bmatrix}$$

Softmax transformation

Multinomial logistic regression

- How to learn weights? maximum likelihood estimation

$$p(y_i = C_k|\boldsymbol{x_i}, \mathbf{W}) = \frac{\exp(\boldsymbol{w}_k^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)}$$

- Let $a_{ik} = \boldsymbol{w}_k^T \boldsymbol{x}_i$ and $p(y_{ik} = 1|\boldsymbol{x}_i, \boldsymbol{W}) = \tilde{y}_{ik}$
- Likelihood function

$$L(\boldsymbol{W}) = \prod_{i=1}^{N} \prod_{k=1}^{K} \mathrm{p}(y_i = C_k|\boldsymbol{x}_i, \boldsymbol{W})^{c_{ik}} = \prod_{i=1}^{N} \prod_{k=1}^{K} \tilde{y}_{\mathrm{ik}}{}^{c_{ik}}$$

only the term corresponding to the true class remains (since raising to the power of 0 gives 1)

- Negative likelihood function

$$NLL(\boldsymbol{W}) = -\sum_{i=1}^{N} \sum_{k=1}^{K} c_{ik} \log(\tilde{y}_{\mathrm{i\,k}})$$

cross-entropy error function for the multiclass classification

$$c_{ik} = \begin{cases} 1, & \text{if } y_i = C_k \\ 0, & \text{otherwise.} \end{cases}$$

$\boldsymbol{c}_i$ is the one-hot encoding of the label (so $c_{ik} = \mathbb{I}(y_i = k)$)

$\mathbb{I}$ – Indicator function

Linear models of classification

Sample Index (i)	Species (class) (y_i)	bill_length_mm $(x_{1,i})$	bill_depth_mm $(x_{2,i})$	flipper_length_mm $(x_{3,i})$	body_mass_g $(x_{4,i})$
1	Adelie	39.1	18.7	181.0	3750.00
2	Chinstrap	46.5	17.9	192.0	3500.00
3	Gentoo	46.1	13.2	211.0	4500.0

- One hot encoder

Sample Index (i)	Species (class) c_{ik}	bill_length_mm $(x_{1,i})$	bill_depth_mm $(x_{2,i})$	flipper_length_mm $(x_{3,i})$	body_mass_g $(x_{4,i})$
1	1 0 0	39.1	18.7	181.0	3750.00
2	0 1 0	46.5	17.9	192.0	3500.00
3	0 0 1	46.1	13.2	211.0	4500.0

Penguins' dataset https://allisonhorst.github.io/palmerpenguins/

Linear models of classification

➢One hot encoder

Sample Index (i)	Species (class) c_{ik}	bill_length_mm $(x_{1,i})$	bill_depth_mm $(x_{2,i})$	flipper_length_mm $(x_{3,i})$	body_mass_g $(x_{4,i})$
1	1 0 0	39.1	18.7	181.0	3750.00
2	0 1 0	46.5	17.9	192.0	3500.00
3	0 0 1	46.1	13.2	211.0	4500.0

$$L(\boldsymbol{W}) = \prod_{i=1}^{N} \prod_{k=1}^{K} \mathrm{p}(y_i = C_k|\boldsymbol{x}_i, \boldsymbol{W})^{c_{ik}} = \prod_{i=1}^{N} \prod_{k=1}^{K} \tilde{y}_{\mathrm{ik}}{}^{c_{ik}}$$

$$= \tilde{y}_{11}{}^{c_{11}} \tilde{y}_{12}{}^{c_{12}} \tilde{y}_{13}{}^{c_{13}} \tilde{y}_{21}{}^{c_{21}} \tilde{y}_{22}{}^{c_{22}} \tilde{y}_{23}{}^{c_{23}} \tilde{y}_{31}{}^{c_{31}} \tilde{y}_{32}{}^{c_{32}} \tilde{y}_{33}{}^{c_{33}}$$

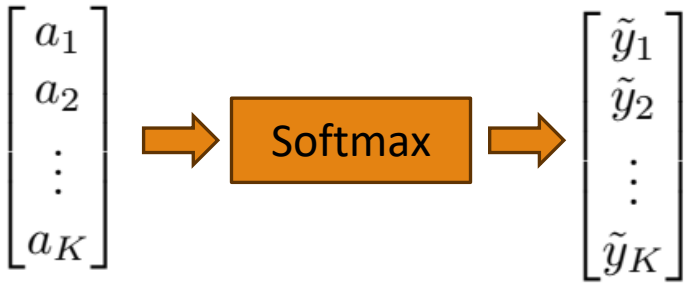
$$= \tilde{y}_{11}{}^{1} \tilde{y}_{12}{}^{0} \tilde{y}_{13}{}^{0} \tilde{y}_{21}{}^{0} \tilde{y}_{22}{}^{1} \tilde{y}_{23}{}^{0} \tilde{y}_{31}{}^{0} \tilde{y}_{32}{}^{0} \tilde{y}_{33}{}^{1}$$

$$p(y_i = C_k|\boldsymbol{x}_i, \mathbf{W}) = \frac{\exp(\boldsymbol{w}_k^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)}$$

$$p(y_{ik} = 1|\boldsymbol{x}_i, \boldsymbol{W}) = \tilde{y}_{ik}$$

$$\tilde{y}_{11} = \frac{\exp(\boldsymbol{w}_1^T \boldsymbol{x}_1)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} \quad \tilde{y}_{12} = \frac{\exp(\boldsymbol{w}_2^T \boldsymbol{x}_1)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} \quad \tilde{y}_{13} = \frac{\exp(\boldsymbol{w}_3^T \boldsymbol{x}_1)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)}$$

Multinomial logistic regression



- Jacobian matrix

$$a_k = \boldsymbol{w}_k^T \boldsymbol{x}_i \qquad \tilde{y}_k = \frac{\exp(\boldsymbol{w}_k{}^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j{}^T \boldsymbol{x}_i)} = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

$$\frac{\partial \tilde{y}_k}{\partial a_j} = \tilde{y}_k \left(\mathbb{I}(k=j) - \tilde{y}_j\right)$$

$$J = \begin{bmatrix} \frac{\partial \tilde{y}_1}{\partial a_1} & \frac{\partial \tilde{y}_1}{\partial a_2} & \cdots & \frac{\partial \tilde{y}_1}{\partial a_K} \\ \frac{\partial \tilde{y}_2}{\partial a_1} & \frac{\partial \tilde{y}_2}{\partial a_2} & \cdots & \frac{\partial \tilde{y}_2}{\partial a_K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \tilde{y}_j}{\partial a_1} & \frac{\partial \tilde{y}_j}{\partial a_2} & \cdots & \frac{\partial \tilde{y}_j}{\partial a_K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \tilde{y}_K}{\partial a_1} & \frac{\partial \tilde{y}_K}{\partial a_2} & \cdots & \frac{\partial \tilde{y}_K}{\partial a_K} \end{bmatrix}$$

For 3 classes Jacobian matrix is given by | Quotient Rule | $\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)f'(x) - f(x)g'(x)}{[g(x)]^2}$

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \tilde{y}_1}{\partial a_1} & \frac{\partial \tilde{y}_1}{\partial a_2} & \frac{\partial \tilde{y}_1}{\partial a_3} \\ \frac{\partial \tilde{y}_2}{\partial a_1} & \frac{\partial \tilde{y}_2}{\partial a_2} & \frac{\partial \tilde{y}_2}{\partial a_3} \\ \frac{\partial \tilde{y}_3}{\partial a_1} & \frac{\partial \tilde{y}_3}{\partial a_2} & \frac{\partial \tilde{y}_3}{\partial a_3} \end{bmatrix} = \begin{bmatrix} \tilde{y}_1(1-\tilde{y}_1) & -\tilde{y}_1\tilde{y}_2 & -\tilde{y}_1\tilde{y}_3 \\ -\tilde{y}_2\tilde{y}_1 & \tilde{y}_2(1-\tilde{y}_2) & -\tilde{y}_2\tilde{y}_3 \\ -\tilde{y}_3\tilde{y}_1 & -\tilde{y}_3\tilde{y}_2 & \tilde{y}_3(1-\tilde{y}_3) \end{bmatrix}$$

$$\mathbf{J} = \frac{\partial \tilde{\boldsymbol{y}}}{\partial \boldsymbol{a}} = (\tilde{\boldsymbol{y}}\mathbf{1}^{\mathsf{T}}) \odot (\mathbf{I} - \mathbf{1}\tilde{\boldsymbol{y}}^{\mathsf{T}})$$

where $\odot$ is elementwise product

When $k = j, \mathbb{I}(k=j)$ =1 else 0. $\tilde{\boldsymbol{y}}\mathbf{1}^{\mathsf{T}}$ copies $\tilde{\boldsymbol{y}}$ across each column, and $\mathbf{1}\tilde{\boldsymbol{y}}^{\mathsf{T}}$ copies $\tilde{\boldsymbol{y}}$ across each row. **I**- identity matrix

Multinomial logistic regression

$$NLL(\boldsymbol{W}) = -\sum_{i=1}^{N}\sum_{k=1}^{K} c_{ik}\log(\tilde{y}_{i\,k})$$

$$\tilde{y}_{ik} = \frac{\exp(\boldsymbol{w}_k{}^T\boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j{}^T\boldsymbol{x}_i)} = \frac{\exp(a_{ik})}{\sum_j \exp(a_{ij})}$$

$$\frac{\partial \tilde{y}_k}{\partial a_j} = \tilde{y}_k(\mathbb{I}(k=j) - \tilde{y}_j)$$

$$a_{ik} = \boldsymbol{w}_k^T\boldsymbol{x}_i$$

- Consider i-th data sample

$$\nabla \boldsymbol{w}_k\, NLL(\boldsymbol{W})_i = -\sum_{k=1}^{K} \frac{\partial NLL(\boldsymbol{W})_i}{\partial \tilde{y}_{i\,k}} \frac{\partial \tilde{y}_{i\,k}}{\partial a_{i\,k}} \frac{\partial a_{i\,k}}{\partial \boldsymbol{w}_k}$$

$$= -\sum_{k=1}^{K} c_{ik}\frac{1}{\tilde{y}_{i\,k}}\tilde{y}_{i\,k}(\mathbb{I}(k=i) - \tilde{y}_{ik})\boldsymbol{x}_i \qquad = \sum_{k=1}^{K}(\tilde{y}_{ik} - c_{ik})\,\boldsymbol{x}_i$$

- For all N data samples and all K classes

$$\mathrm{g}(\boldsymbol{W}) = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{x}_i(\tilde{\boldsymbol{y}}_i - \boldsymbol{c}_i)^{\boldsymbol{T}}$$

1x K vector

(D+1) x 1 vector

(D+1) × K matrix

When $k = j, \mathbb{I}(k=j)$ =1 else 0.

Multinomial logistic regression

- Stochastic gradient descent

$$g(\boldsymbol{W}) = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{x}_i(\tilde{\boldsymbol{y}}_i - \boldsymbol{c}_i)^{\boldsymbol{T}}$$

- Update of weight (i is the sample index)

$$\boldsymbol{W}_{i+1} \leftarrow \boldsymbol{W}_i - \alpha \boldsymbol{x}_i(\tilde{\boldsymbol{y}}_i - \boldsymbol{c}_i)^{\boldsymbol{T}}$$

- Batch gradient descent

$$\boldsymbol{W}_{\boldsymbol{new}} \leftarrow \boldsymbol{W}_{\boldsymbol{old}} - \alpha \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{x}_i(\tilde{\boldsymbol{y}}_i - \boldsymbol{c}_i)^{\boldsymbol{T}}$$

Multinomial logistic regression

- Hessian of the NLL for multinomial logistic regression is given by

$$H(\boldsymbol{W}) = \frac{1}{N}\sum_{i=1}^{N} \left(\text{diag}\left(\tilde{\boldsymbol{y}}_i\right) - \tilde{\boldsymbol{y}}_i\tilde{\boldsymbol{y}}_{\boldsymbol{i}}^{\boldsymbol{T}}\right) \otimes \boldsymbol{x}_i\,\boldsymbol{x}_{\boldsymbol{i}}^{\boldsymbol{T}}$$

- To develop a batch algorithm for the multiclass problem, we use the Newton-Raphson update.
- The IRLS algorithm involves evaluating the Hessian matrix

⊗ Kronecker product

Probabilistic view of classification

- Probabilistic view of classification
  - **Discriminative classifier**
    - Directly fit the class posterior $p(y_i = C_k|\boldsymbol{x_i}, \boldsymbol{\theta})$
    - E.g., logistic regression, multi class logistic regression
  - **Generative classifier**
    - Model how to generate data using the conditional density $p(\boldsymbol{x_i}|y_i = Ck)$ and class prior probability $p(y_i = C_k)$. Then using Bayes rule

Bayes rule

$$p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)}$$

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = C_k|\boldsymbol{\theta})p(\mathbf{x}|y = C_k, \boldsymbol{\theta})}{\sum_{C_k'} p(y = C_k'|\boldsymbol{\theta})p(\mathbf{x}|y = C_k', \boldsymbol{\theta})}$$

Recall: Bayes rule

Suppose an individual is scheduled to undergo a medical examination, and in the event of a positive outcome, the individual will be diagnosed with disease "A". The test's sensitivity is 90%, indicating that if the disease is present, there is a likelihood of 0.9 for the medical examination to produce a positive result. It is given that the probability of having the disease is 0.0001. Further, it is given that out of the 500 individuals who faced the aforementioned medical assessment, 50 yielded positive results without having the disease. If we denote an individual with the disease "A" as "$a = 1$" and a positive medical examination result as "$t = 1$".
What are the values of $p(t = 1|a = 1)$, $p(a = 1)$ and $p(t = 1|a = 0)$. Here, $p(x)$ indicates the probability of event x.

If the medical examination yields a positive result, what is the probability of actually having the disease ?

$p(t = 1|a = 1) = 0.90$, $p(a = 1)$=0.0001, and $p(t = 1|a = 0) = 50/500 = 0.1$.

Recall: Bayes rule

Using Bayes' theorem.

$$
\begin{aligned}
p(a=1|t=1) &= \frac{p(t=1|a=1)p(a=1)}{p(t=1)}, \\
&= \frac{p(t=1|a=1)p(a=1)}{p(t=1|a=1)p(a=1)+p(t=1|a=0)p(a=0)}, \\
&= \frac{0.90\times 0.0001}{0.90\times 0.0001+0.1\times(1-0.0001)} = 0.0009 = 0.09\%.
\end{aligned}
$$

Generative models

- Generative classifier

generate the features $\boldsymbol{x}$ for each class

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = Ck|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta})}{\sum_{C_k'} p(y = C_k'|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k', \boldsymbol{\theta})}$$

For two classes

$$p(y = C_1|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})}{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta}) + p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})}$$

$$= \frac{\mathbf{1}}{1+p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})/p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})} = \frac{\mathbf{1}}{1+\exp(-a(\boldsymbol{x}))}$$

$$= \text{sigm}(a(\boldsymbol{x}))= \sigma(a(\boldsymbol{x}))$$

logistic sigmoid function

squashing function

equivalent form of the posterior probabilities

$$a(\boldsymbol{x}) = \frac{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})}{p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})}$$

For multi-class problem it is SoftMax function

Generative models

- Generative classifier

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = C_k|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta})}{\sum_{C_k'} p(y = C_k'|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k', \boldsymbol{\theta})}$$

generate the features $\boldsymbol{x}$ for each class

- For two classes

$$p(y = C_1|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})}{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta}) + p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})}$$

- Linear discriminant analysis* $\log p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \tilde{\boldsymbol{w}}^T\boldsymbol{x} + \text{const}$ (linear function of $\boldsymbol{x}$)

*Although names says discriminant this is a generative model

Generative models: Continuous inputs

- Class-conditional densities are Gaussian distributed

$$p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T {\Sigma_k}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

Multi−Variate Gaussian

$\boldsymbol{\mu}_k$ = mean vector

Σ_k= covariance matrix

- Assume that **same covariance matrix** is shared with all classes

$$p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

Recall:The Multivariate Gaussian

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu},\Sigma) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right]$$

- Consider simple case D=2

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \qquad \boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix} \qquad \Sigma = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$$

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu},\Sigma) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right] = \frac{1}{(2\pi)\ (\sigma_1^2\sigma_2^2-0)^{1/2}} \exp\left[-\frac{1}{2}\begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}^T \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}^{-1} \begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}\right]$$

$$= \frac{1}{(2\pi\sigma_1\ \sigma_1\ )} \exp\left[-\frac{1}{2}\begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}^T \begin{bmatrix} 1/\sigma_1^2 & 0 \\ 0 & 1/\sigma_2^2 \end{bmatrix} \begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}\right]$$

$$= \frac{1}{(2\pi\sigma_1\ \sigma_1\ )} \exp\left[-\frac{1}{2}\begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}^T \begin{bmatrix} 1/\sigma_1^2(x_1-\mu_1) \\ 1/\sigma_2^2(x_2-\mu_2) \end{bmatrix}\right] = \frac{1}{(2\pi)^{1/2}\ \sigma_1} \exp\left[-\frac{1}{2\sigma_1^2}(x_1-\mu_1)^2\right] \frac{1}{(2\pi)^{1/2}\ \sigma_2} \exp\left[-\frac{1}{2\sigma_2^2}(x_2-\mu_2)^2\right]$$

product of two independent Gaussian densities
(if cov(x_1, x_2) and cov (x_2, x_1) are zero)

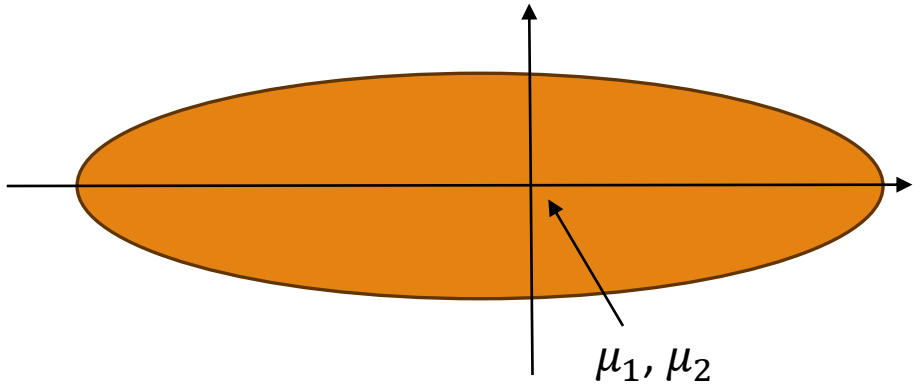
Recall:The Multivariate Gaussian

Defining $r_1 = \sqrt{2\sigma_1^2 \log\left(\frac{1}{2\pi p \sigma_1 \sigma_2}\right)}$ $r_2 = \sqrt{2\sigma_2^2 \log\left(\frac{1}{2\pi p \sigma_1 \sigma_2}\right)}$

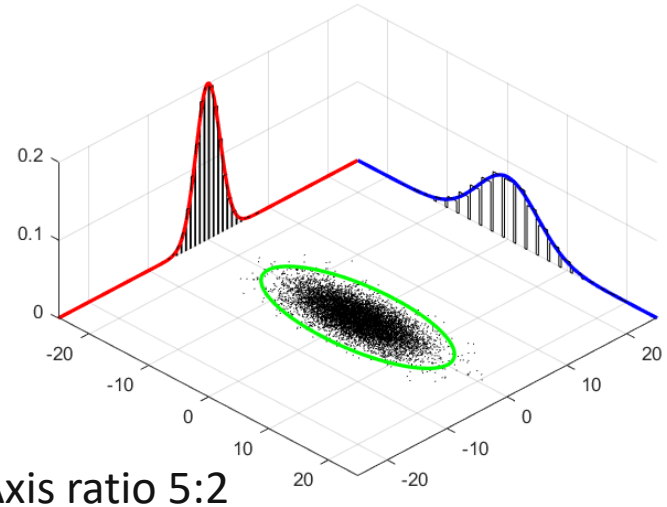
$$p = \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2\sigma_1^2}(x_1 - \mu_1)^2 - \frac{1}{2\sigma_2^2}(x_2 - \mu_2)^2\right)$$

$$1 = \left(\frac{x_1 - \mu_1}{r_1}\right)^2 + \left(\frac{x_2 - \mu_2}{r_2}\right)^2$$

- axis-aligned ellipse
- center (μ_1, μ_2)
- x_1 axis has length 2 r_1
- x_2 axis has length 2 r_2

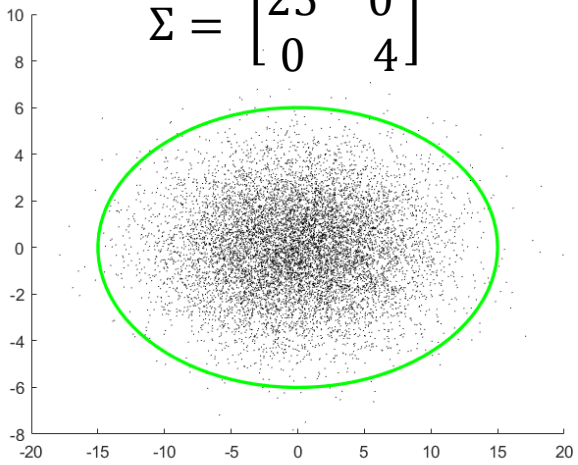


$$\Sigma = \begin{bmatrix} 25 & 0 \\ 0 & 4 \end{bmatrix} \quad \boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

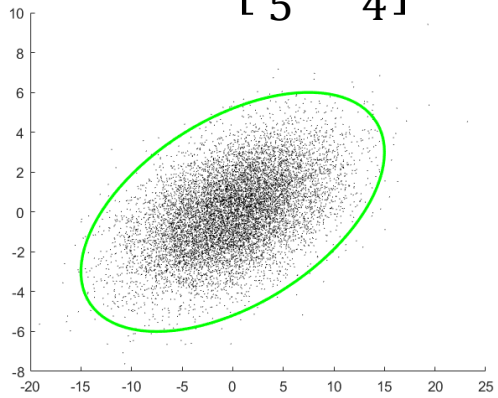


Axis ratio 5:2

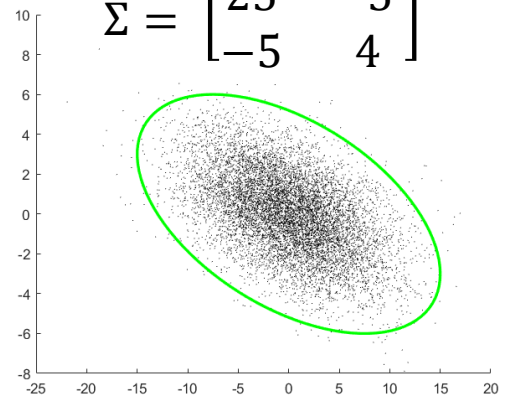
$$\Sigma = \begin{bmatrix} 25 & 0 \\ 0 & 4 \end{bmatrix}$$



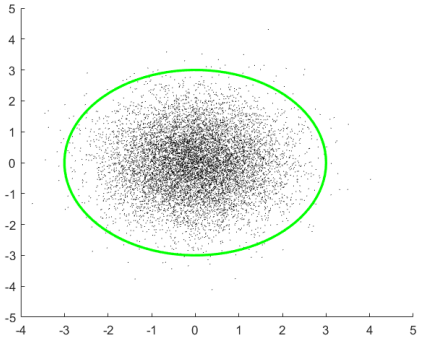
$$\Sigma = \begin{bmatrix} 25 & 5 \\ 5 & 4 \end{bmatrix}$$



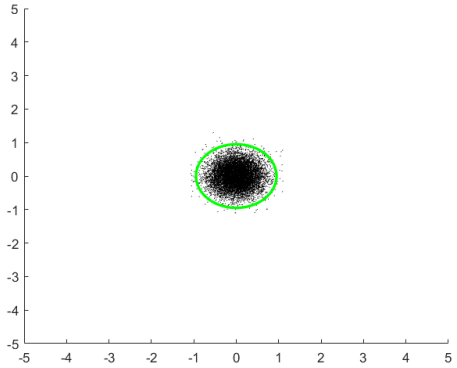
$$\Sigma = \begin{bmatrix} 25 & -5 \\ -5 & 4 \end{bmatrix}$$



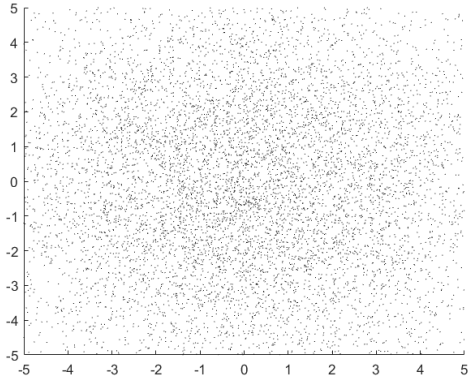
Recall:The Multivariate Gaussian



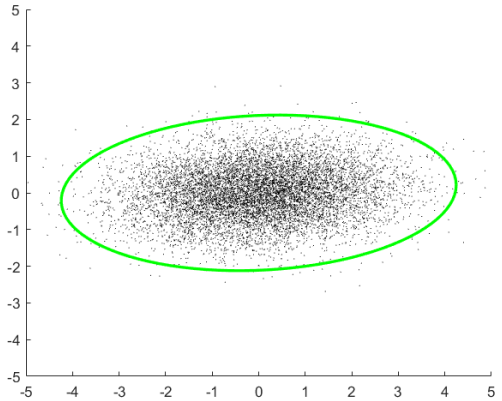
$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$$

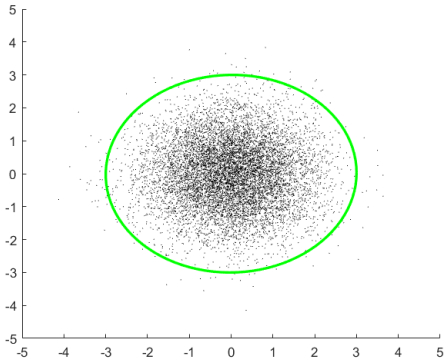


$$\Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$$

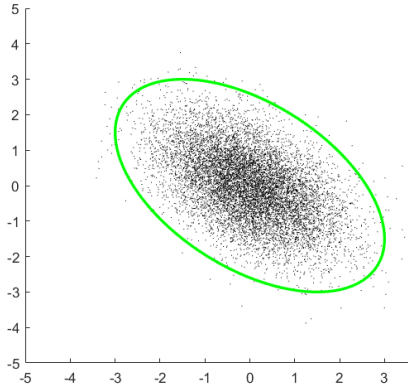


$$\Sigma = \begin{bmatrix} 2 & 0.1 \\ 0.1 & 0.5 \end{bmatrix}$$

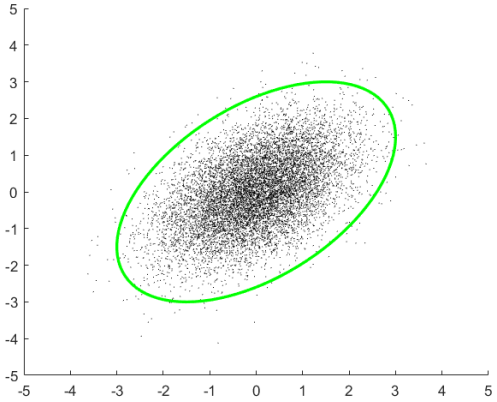
$$\boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



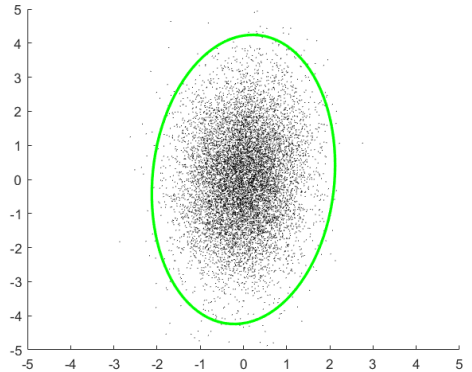
$$\Sigma = \begin{bmatrix} 1 & 0.01 \\ 0.01 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 0.5 & 0.1 \\ 0.1 & 2 \end{bmatrix}$$

When the off-diagonal elements are high, it implies that changes in one variable are strongly associated with changes in another variable.

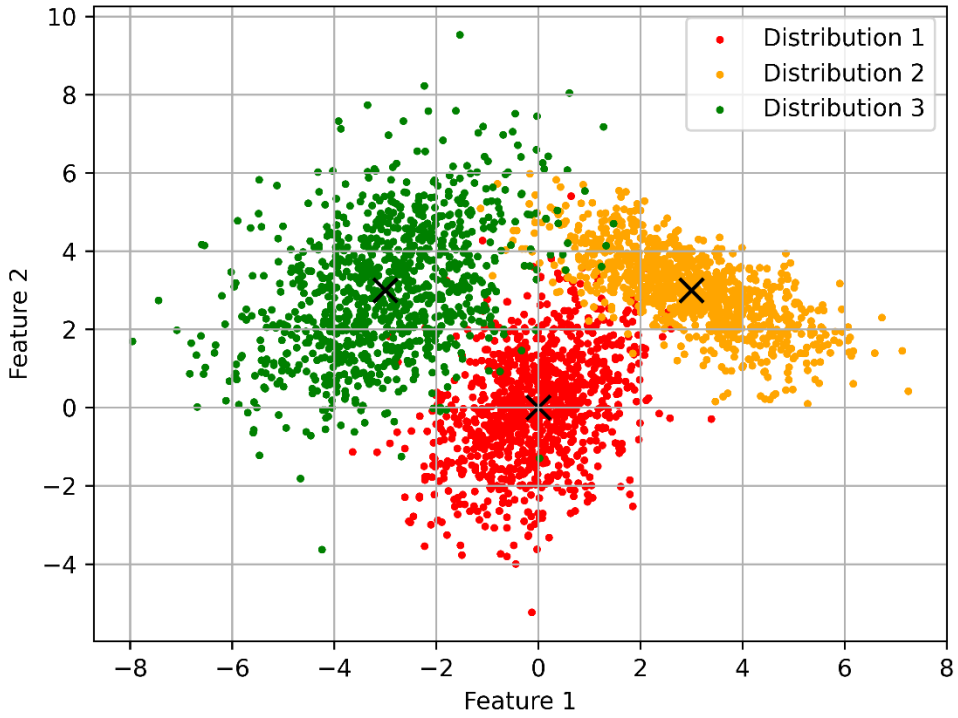
Generative models: Continuous inputs

- Class-conditional densities are Gaussian distributed

$$p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T {\Sigma_k}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

Multi−Variate Gaussian

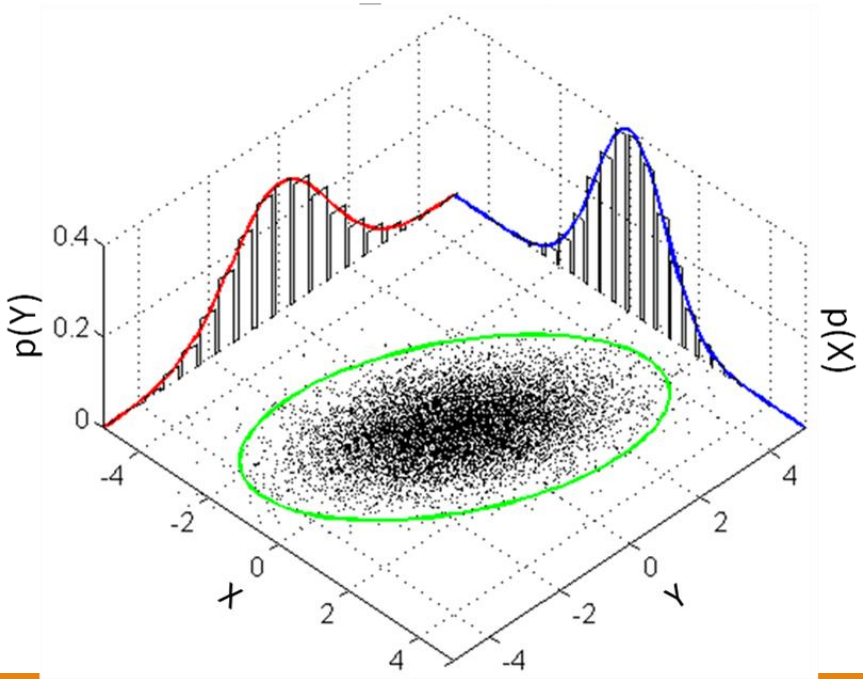
$\boldsymbol{\mu}_k$ = mean vector



$$\mu_1 = \begin{bmatrix}0\\0\end{bmatrix}, \Sigma_1 = \begin{bmatrix}1 & 0.5\\0.5 & 2\end{bmatrix}$$

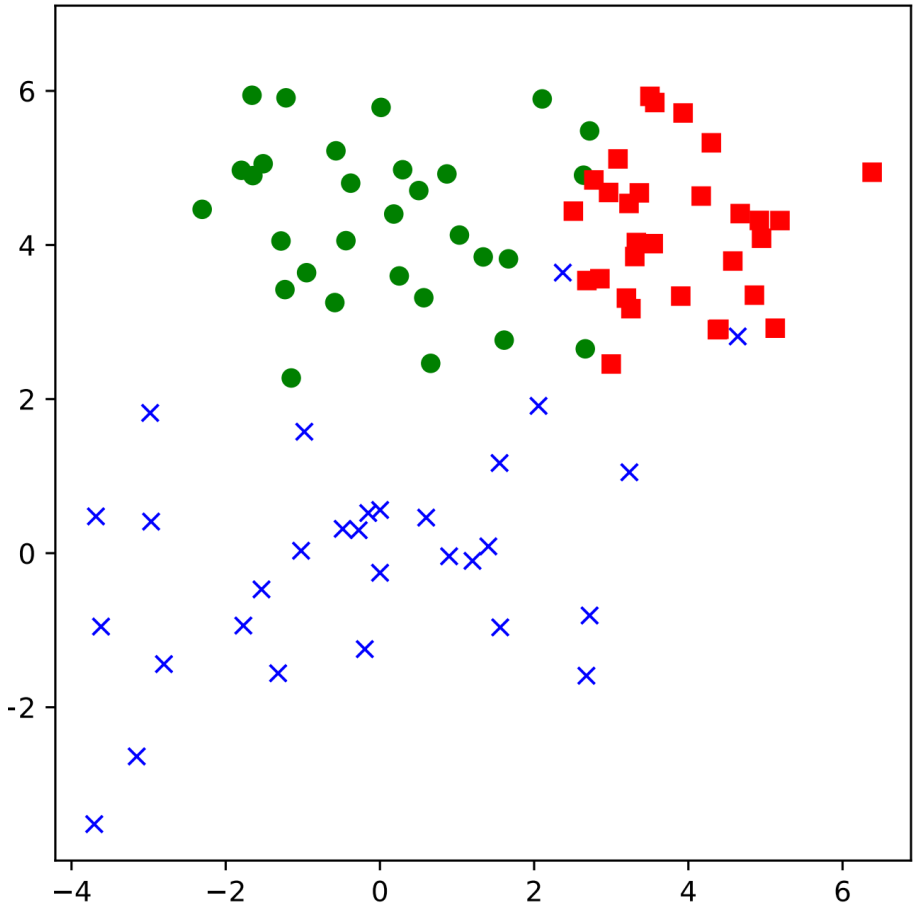
$$\mu_2 = \begin{bmatrix}3\\3\end{bmatrix}, \Sigma_2 = \begin{bmatrix}2 & -1\\-1 & 1\end{bmatrix}$$

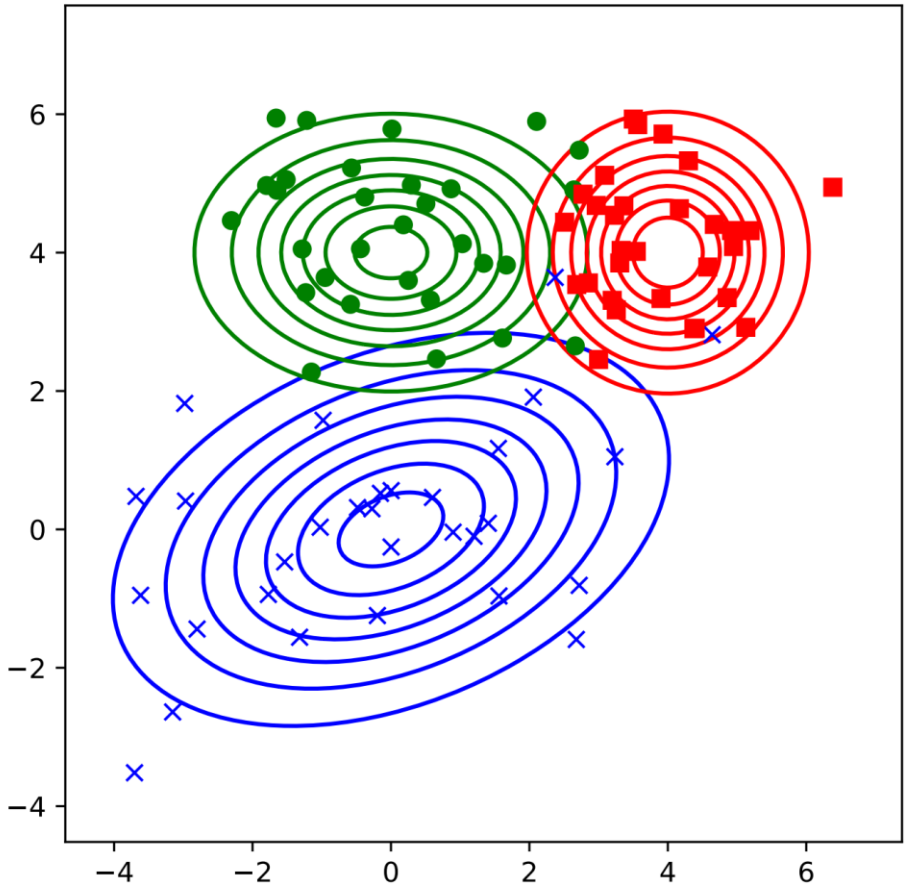
$$\mu_3 = \begin{bmatrix}-3\\3\end{bmatrix}, \Sigma_3 = \begin{bmatrix}2 & 1\\1 & 3\end{bmatrix}$$



MultivariateNormal - Multivariate normal distribution - Wikipedia

Generative models: Continuous inputs





Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*, 2012.
probml.github.io/notebooks#discrim_analysis_dboundaries_plot2.ipynb

Generative models: Continuous inputs

- Class-conditional densities are Gaussian distributed

$$p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

Multi−Variate Gaussian

$\boldsymbol{\mu}_k$ = mean vector

Σ= covariance matrix

- For two classes

$$\mathrm{p}(y = C_1|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})}{p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta}) + p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})}$$

independent of C_1

prior probability of class C_1

$p(y = C_1|\boldsymbol{\theta}) = \pi_1$

$$\log(\mathrm{p}(y = C_1|\mathbf{x}, \boldsymbol{\theta})) = \log p(y = C_1|\boldsymbol{\theta}) + \log(p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta})) + \text{constant}$$

$$= \log p(y = C_1|\boldsymbol{\theta}) + \log(\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_1, \Sigma)) + \text{constant}$$

$$= \log p(y = C_1|\boldsymbol{\theta}) + -\frac{1}{2}\log(|\Sigma|) - \frac{D}{2}\log(2\pi)\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_1)\right] + \text{constant}$$

$$= \log \pi_1 - \frac{1}{2}\log(|\Sigma|) - \frac{D}{2}\log(2\pi)\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_1)\right] + \text{constant}$$

C_k class label, for binary classes it is $C_k \in \{1,0\}$

Generative models: Continuous inputs

- Class-conditional densities are Gaussian distributed

$$p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k)\right]$$

Multi−Variate Gaussian

$\boldsymbol{\mu}_k$ = mean vector

Σ= covariance matrix

$$\log\left(\mathrm{p}(y = C_k|\mathbf{x}, \boldsymbol{\theta})\right) = \log p(y = C_k|\boldsymbol{\theta}) + \log(\, p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta})) + \text{constant}$$

$$= \log \pi_k - \frac{1}{2}\log(|\Sigma|) - \frac{D}{2}\log(2\pi)\left[-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k)\right] + \text{constant}$$

$$= \log \pi_k - \frac{1}{2}\boldsymbol{\mu}_k{}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{\mu}_k + \boldsymbol{x}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{\mu}_k + \text{constant} - \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{x}$$

$$= \log \pi_k - \frac{1}{2}\boldsymbol{\mu}_k{}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{\mu}_k + \boldsymbol{x}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{\mu}_k + \text{constant} - \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{x}$$

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \sigma\big(a(\boldsymbol{x})\big) \Rightarrow \log\left(\mathrm{p}(y = (y = Ck|\mathbf{x}, \boldsymbol{\theta})|\mathbf{x}, \boldsymbol{\theta})\right) = (\tilde{\boldsymbol{w}}^T\boldsymbol{x} \quad + \mathrm{w}_0)$$

$$\tilde{\boldsymbol{w}} = \Sigma^{-1}\boldsymbol{\mu}_k$$

$$\mathrm{w}_0 = \log \pi_k - \frac{1}{2}\boldsymbol{\mu}_k{}^{\mathrm{T}}\Sigma^{-1}\boldsymbol{\mu}_k$$

Due to shared covariance matrix assumption, the quadratic part $\boldsymbol{x}^T\Sigma^{-1}\boldsymbol{x}$ cancels off and $\log\left(\mathrm{p}(y = C_k|\boldsymbol{x}, \boldsymbol{\theta})\right)$ is a linear function of $\boldsymbol{x}$. This is called linear discriminant analysis.

Generative models: Continuous inputs

- Maximum likelihood estimation

$$p(\boldsymbol{x}|y = Ck, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_k}, \boldsymbol{\Sigma}) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = Ck|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k, \boldsymbol{\theta})}{\sum_{C_k'} p(y = C_k'|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_k', \boldsymbol{\theta})} \qquad p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) \propto p(y = Ck|\boldsymbol{\theta})p(\boldsymbol{x}|y = Ck, \boldsymbol{\theta})$$

- For two classes

$$p(y = C_1|\mathbf{x}, \boldsymbol{\theta}) \propto p(y = C_1|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta}) \qquad p(y = C_2|\mathbf{x}, \boldsymbol{\theta}) \propto p(y = C_2|\boldsymbol{\theta})p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})$$

$$p(y = C_1|\mathbf{x}, \boldsymbol{\theta}) \propto \pi_1\, p(\boldsymbol{x}|y = C_1, \boldsymbol{\theta}) \qquad p(y = C_2|\mathbf{x}, \boldsymbol{\theta}) \propto (1-\pi_1)\, p(\boldsymbol{x}|y = C_2, \boldsymbol{\theta})$$

$$p(y = C_1|\mathbf{x}, \boldsymbol{\theta}) \propto \pi_1\, \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma}) \qquad p(y = C_2|\mathbf{x}, \boldsymbol{\theta}) \propto (1-\pi_1)\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})$$

- likelihood function

$$p(\boldsymbol{c}|\pi_1, \boldsymbol{\mu_2}, \boldsymbol{\mu_1}, \boldsymbol{\Sigma}) = \mathrm{L} = \prod_i [p(y = C_1|\mathbf{x_i}, \boldsymbol{\theta})]^{c_i} [p(y = C_2|\mathbf{x_i}, \boldsymbol{\theta})]^{1-c_i} = \prod_i [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})]^{c_i} [(1-\pi_1)\mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]^{1-c_i}$$

c_i class label of i-th sample, for binary case it is $c_i \in \{1,0\}$

Generative models: Continuous inputs

- Likelihood function

$$p(\boldsymbol{c}|\pi_1, \boldsymbol{\mu_2}, \boldsymbol{\mu_1}, \boldsymbol{\Sigma}) = \mathrm{L} = \prod_i [p(y = C_1|\mathbf{x_i}, \boldsymbol{\theta})]^{c_i} [p(y = C_2|\mathbf{x_i}, \boldsymbol{\theta})]^{1-c_i} = \prod_i [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})]^{c_i} [(1-\pi_1)\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]^{1-c_i}$$

- Log likelihood function $\sum_i c_i \log [\pi_1 \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})] + (1 - c_i) \log [(1-\pi_1) \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]$

- Maximize with respect to π_1 $\sum_i c_i (\log [\pi_1] + \log[\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})]) + (1 - c_i) (\log[1-\pi_1] + \log[\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})])$

In class activity, show $\mathrm{L}\pi_1 = \sum_i c_i \log [\pi_1] + (1 - c_i) \log [(1-\pi_1)]$

$$\frac{\partial \mathrm{L}\pi_1}{\partial \pi_1} = 0 \qquad \pi_1 = \frac{\mathrm{N}_1}{N} = \frac{\mathrm{N}_1}{\mathrm{N}_1 + \mathrm{N}_2}$$

Generative models: Continuous inputs

- Likelihood function

$$p(\boldsymbol{c}|\pi_1, \boldsymbol{\mu_2}, \boldsymbol{\mu_1}, \boldsymbol{\Sigma}) = \mathrm{L} = \prod_i [p(y = C_1|\mathbf{x_i}, \boldsymbol{\theta})]^{c_i} [p(y = C_2|\mathbf{x_i}, \boldsymbol{\theta})]^{1-c_i} = \prod_i [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})]^{c_i} [(1-\pi_1)\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]^{1-c_i}$$

- Log likelihood function

$$\sum_i c_\mathrm{i} \log [\pi_1 \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})] + (1 - c_\mathrm{i}) \log [(1-\pi_1) \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]$$

- Maximize with respect to $\boldsymbol{\mu_1}$

$$\mathrm{L}\boldsymbol{\mu_1} = \sum_i c_\mathrm{i} \log [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})] = \sum_i c_\mathrm{i} \left[-\frac{1}{2}(\mathbf{x_i} - \boldsymbol{\mu}_1)^T \Sigma^{-1} (\mathbf{x_i} - \boldsymbol{\mu}_1)\right] + const$$

$$\frac{\partial \mathrm{L}\boldsymbol{\mu_1}}{\partial \boldsymbol{\mu_1}} = 0 \qquad \boldsymbol{\mu_1} = \frac{1}{\mathrm{N}_1} \sum_{i=1}^{N} c_\mathrm{i}\, \mathbf{x_i} \qquad \text{Similarly,} \qquad \boldsymbol{\mu_2} = \frac{1}{\mathrm{N}_2} \sum_{i=1}^{N} (1 - c_\mathrm{i})\, \mathbf{x_i}$$

Generative models: Continuous inputs

- Likelihood function

$$p(\boldsymbol{c}|\pi_1, \boldsymbol{\mu_2}, \boldsymbol{\mu_1}, \boldsymbol{\Sigma}) = \mathrm{L} = \prod_i [p(y = C_1|\mathbf{x_i}, \boldsymbol{\theta})]^{c_i} [p(y = C_2|\mathbf{x_i}, \boldsymbol{\theta})]^{1-c_i} = \prod_i [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})]^{c_i} [(1-\pi_1)\mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]^{1-c_i}$$

- Log likelihood function

$$\sum_i c_\mathrm{i} \log [\pi_1 \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})] + (1 - c_\mathrm{i}) \log [(1-\pi_1) \mathcal{N}(\mathbf{x}|\boldsymbol{\mu_2}, \boldsymbol{\Sigma})]$$

- Maximize with respect to $\boldsymbol{\Sigma}$

$$\mathrm{L}_{\boldsymbol{\Sigma}} = \sum_i c_\mathrm{i} \log [\pi_1 \mathcal{N}(\mathbf{x_i}|\boldsymbol{\mu_1}, \boldsymbol{\Sigma})] = \sum_i -\frac{1}{2} c_\mathrm{i} \log(|\Sigma|) - \left[\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right] + const$$

$$\frac{\partial \mathrm{L}_{\boldsymbol{\Sigma}}}{\partial \boldsymbol{\Sigma}} = 0 \qquad \boldsymbol{\Sigma} = \frac{1}{N} \sum_{i=1}^{N} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{N} \left(\sum_{i=1}^{N} \mathbf{x}_i \mathbf{x}_i^T \right) - \bar{\mathbf{x}} \bar{\mathbf{x}}^T$$

Generative models: Continuous inputs

- Maximum likelihood estimation

$$\hat{\pi}_k = \frac{\mathrm{N_k}}{N} \qquad \widehat{\boldsymbol{\mu_k}} = \frac{1}{\mathrm{N}_k} \sum_{i:\, yi=k}^{N} \mathbf{x_i} \qquad \widehat{\boldsymbol{\Sigma_k}} = \frac{1}{\mathrm{N}_k} \sum_{i:y_i=k}^{N} (\mathbf{x}_i - \boldsymbol{\mu_k})\,(\mathbf{x}_i - \boldsymbol{\mu_k})^T$$

For tied variance ($\boldsymbol{\Sigma} = \boldsymbol{\Sigma_k}$)

$$\widehat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{N}\left(\sum_{i=1}^{N}\mathbf{x}_i\mathbf{x}_i^T\right) - \bar{\mathbf{x}}\bar{\mathbf{x}}^T$$

Generative models: Continuous inputs

- Quadrature discriminant analysis
- We drop the shared covariance matrix assumption

$$p(\boldsymbol{x}|y = Ck, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \triangleq \frac{1}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T {\Sigma_k}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]$$

Multi−Variate Gaussian

$\boldsymbol{\mu}_k$ = mean vector

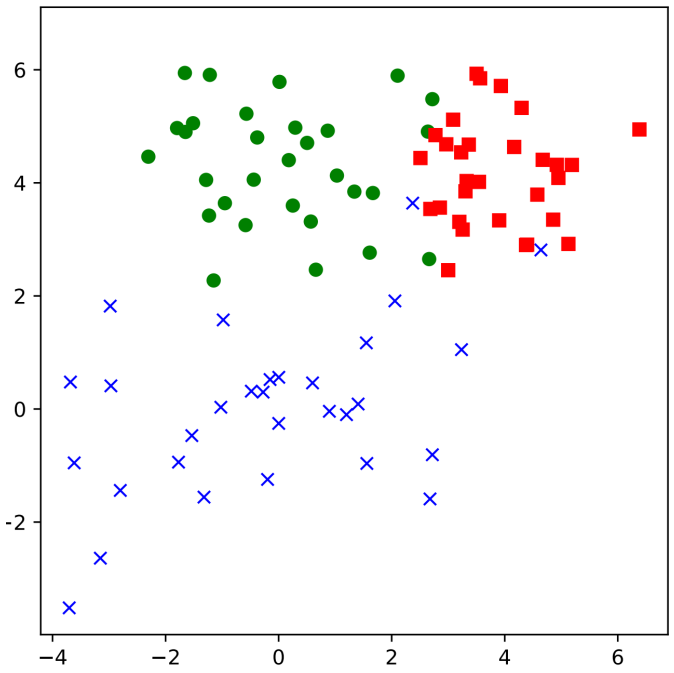
Σ_k= covariance matrix

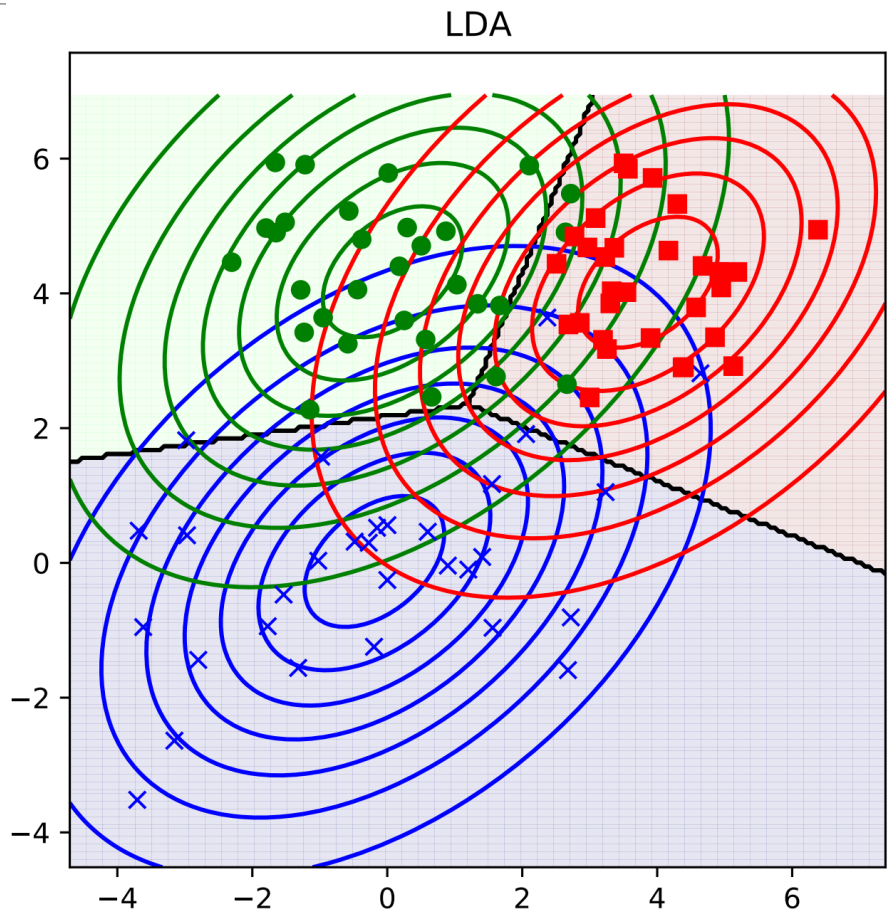
$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = Ck|\boldsymbol{\theta})p(\mathbf{x}|y = C_k, \boldsymbol{\theta})}{\sum_{C_k{}'} p(y = C_k{}'|\boldsymbol{\theta})p(\mathbf{x}|y = C_k{}', \boldsymbol{\theta})}$$

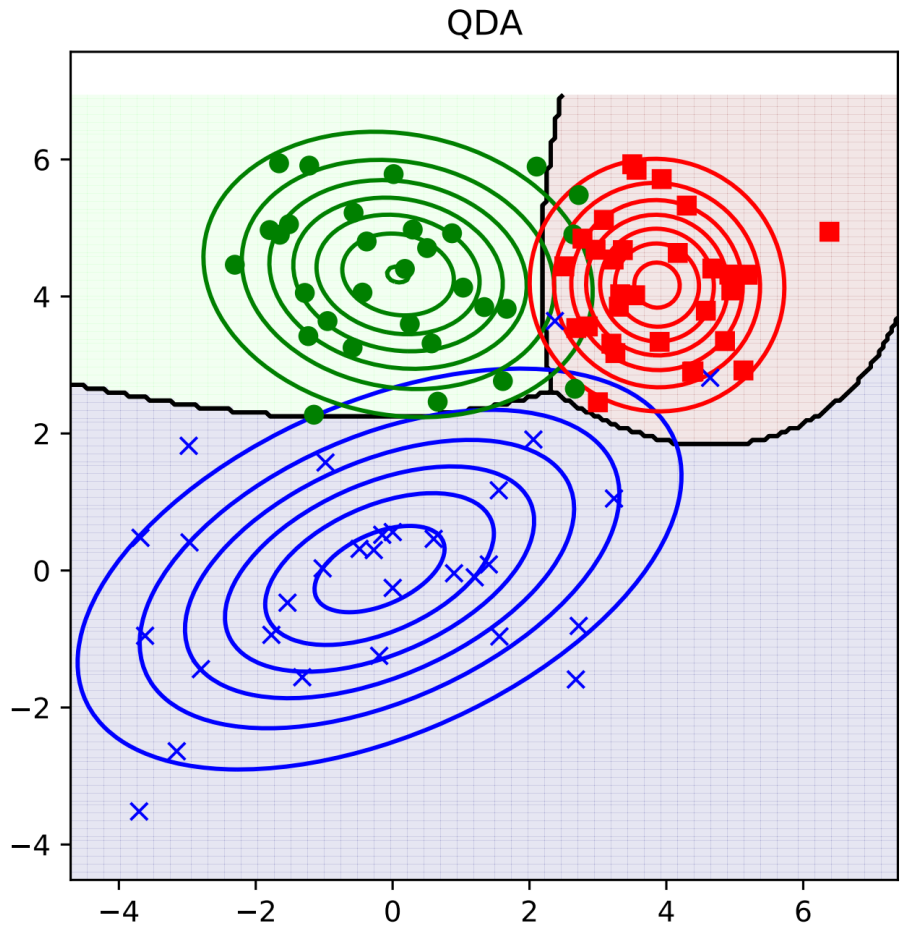
Gaussian discriminant analysis or **GDA**

$$p(y = C_k|\mathbf{x}, \boldsymbol{\theta}) = \frac{\frac{1}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T {\Sigma_k}^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right]}{\sum_{C_k{}'} \pi_{k'}|2\pi\Sigma_{k'}|^{-1/2} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_{k'})^T \Sigma_{k'}^{-1}(\mathbf{x} - \boldsymbol{\mu}_{k'})\right]}$$

QDA & LDA

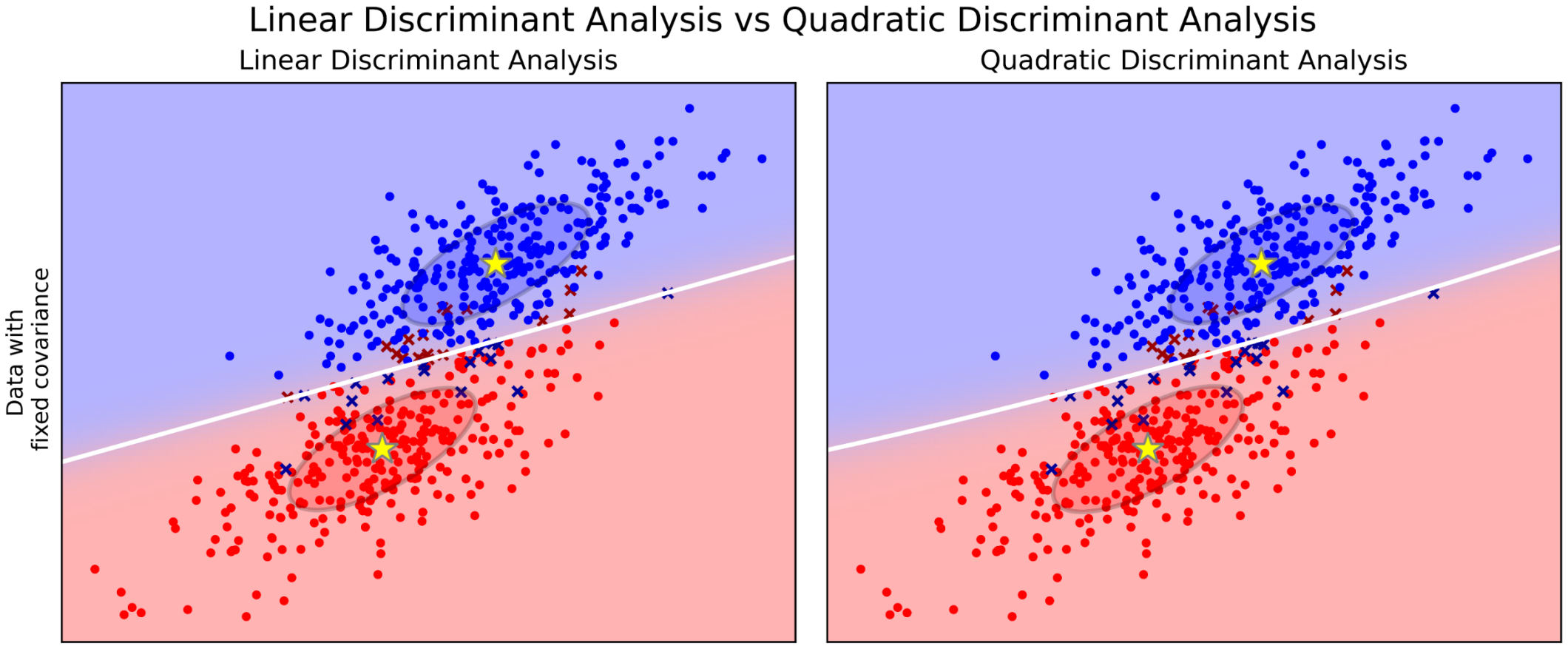






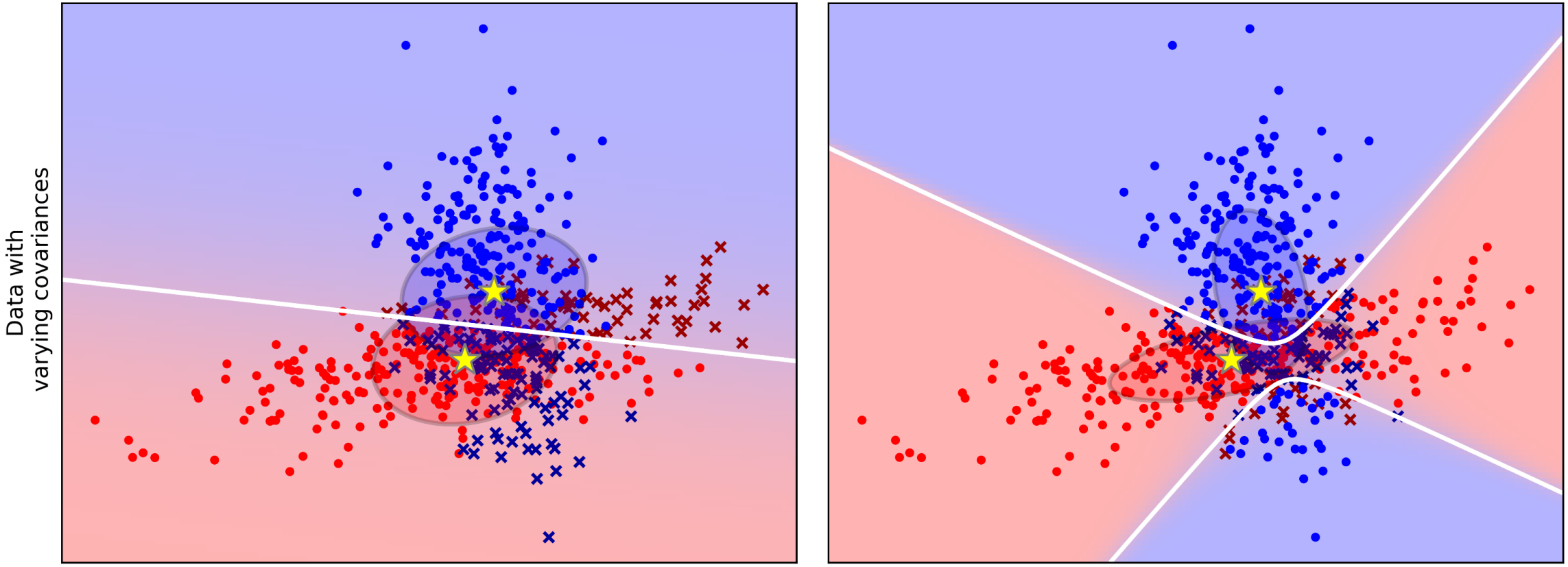
Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*, 2012.
probml.github.io/notebooks#discrim_analysis_dboundaries_plot2.ipynb

QDA & LDA



1.2. Linear and Quadratic Discriminant Analysis — scikit-learn 1.3.0 documentation

QDA & LDA



1.2. Linear and Quadratic Discriminant Analysis — scikit-learn 1.3.0 documentation

Advantages of generative classifiers

- Ease of fitting
- Handling missing features
- Class-specific learning
- Can handle unlabeled data
- Robustness to spurious features

Example:

Consider a dataset consisting of three flower classes: Iris Setosa, Iris Versicolor, and Iris Virginica. In this dataset each data sample is represented by a 4-dimensional vector. After the learning process, a linear classifier is given as follows:

$$\boldsymbol{W} = \begin{bmatrix} 0.41 & 1.46 & -2.26 & -1.02 \\ 0.42 & -1.61 & 0.57 & -1.40 \\ -1.70 & -1.53 & 2.47 & 2.55 \end{bmatrix}^T, \text{ and } \boldsymbol{b} = \begin{bmatrix} 0.26 & 1.09 & -1.21 \end{bmatrix}^T.$$

Suppose that three data samples $\boldsymbol{x}_1$, $\boldsymbol{x}_2$, and $\boldsymbol{x}_3 \in \mathbb{R}^{4\times 1}$ are fed to this linear classifier. Data samples are given by

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_1 & \boldsymbol{x}_2 & \boldsymbol{x}_3 \end{bmatrix} = \begin{bmatrix} 5.1 & 6.0 & 6.3 \\ 3.5 & 2.2 & 3.3 \\ 1.4 & 4.0 & 6.0 \\ 0.2 & 1.0 & 2.5 \end{bmatrix} \in \mathbb{R}^{4\times 3}.$$

Output of the linear classifier for $i-$th data sample is given by $\boldsymbol{y}_i = f\left(\boldsymbol{W}^T\boldsymbol{x}_i + \boldsymbol{b}\right) \in \mathbb{R}^{3\times 1}$. Here, $f(\cdot)$ is the sigmoid function.

i. Compute the output of the linear classifier for three data samples $\boldsymbol{x}_1$, $\boldsymbol{x}_2$, and $\boldsymbol{x}_3$.

ii. Based on the above answer, determine data samples $\boldsymbol{x}_1$, $\boldsymbol{x}_2$, and $\boldsymbol{x}_3$ are either within the same class or belong to different classes? Justify your answer.

iii. Suppose, we need a probabilistic output in (i), is sigmoid function suitable? If not propose required modification(s).

iv. Can we have **4 x 2** matrix for **W ?**

(i) $$\boldsymbol{Y} = \begin{bmatrix} \boldsymbol{y}_1 & \boldsymbol{y}_2 & \boldsymbol{y}_3 \end{bmatrix} = \begin{bmatrix} f\left(\boldsymbol{W}^T\boldsymbol{x}_1 + \boldsymbol{b}\right) & f\left(\boldsymbol{W}^T\boldsymbol{x}_2 + \boldsymbol{b}\right) & f\left(\boldsymbol{W}^T\boldsymbol{x}_3 + \boldsymbol{b}\right) \end{bmatrix},$$

$$= \begin{bmatrix} 0.9800 & 0.0160 & 0.00021 \\ 0.1300 & 0.7329 & 0.16892 \\ 0.0001 & 0.0834 & 0.98491 \end{bmatrix}.$$

Iris Setosa | Iris Versicolor, | Iris Virginica

(ii)

For data sample $\boldsymbol{x}_1$: The highest value is the first one.
For data sample $\boldsymbol{x}_2$: The highest value is the second one.
For data sample $\boldsymbol{x}_3$:The highest value is the third one.
Thus, the data samples $\boldsymbol{x}_1$, $\boldsymbol{x}_2$, and $\boldsymbol{x}_3$ belong to different classes.

(iii) Need to do column-wise normalization such that the total of each column is 1

$$\begin{bmatrix} 0.9800 & 0.0160 & 0.00021 \\ 0.1300 & 0.7329 & 0.16892 \\ 0.0001 & 0.0834 & 0.98491 \end{bmatrix}$$

0.98/(0.98+0.13+0.0001)
0.13/(0.98+0.13+0.0001)
0.0001/(0.98+0.13+0.0001)

Example:

Table Q1.1 displays feature values x_1 and x_2 of data samples which belong to two classes, namely, c_1 and c_2. Note that feature values are rounded to the nearest integer. Here, it is assumed that class-conditional densities are Gaussian distributed, i.e., $p(\boldsymbol{x}|y = c_k, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$. Calculate following.

i. Prior probabilities of two classes $p(y = c_1|\boldsymbol{\theta})$ and $p(y = c_2|\boldsymbol{\theta})$.

ii. Mean vectors $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ of two classes.

Table Q1.1: Feature values x_1 and x_2 of data samples and class labels.

Index (i)	1	2	3	4	5	6	7	8	9	10
$x_{1,i}$	4	2	-5	3	3	-7	-11	-8	-13	4
$x_{2,i}$	1	-3	7	4	8	-13	-6	-14	-10	6
Class label y_i	c_1	c_1	c_1	c_1	c_1	c_2	c_2	c_2	c_2	c_1

$$p(y = c_1|\boldsymbol{\theta}) = \frac{N_1}{N_1 + N_2} = 0.6 \text{ and } p(y = c_2|\boldsymbol{\theta}) = \frac{N_2}{N_1 + N_2} = 0.4.$$

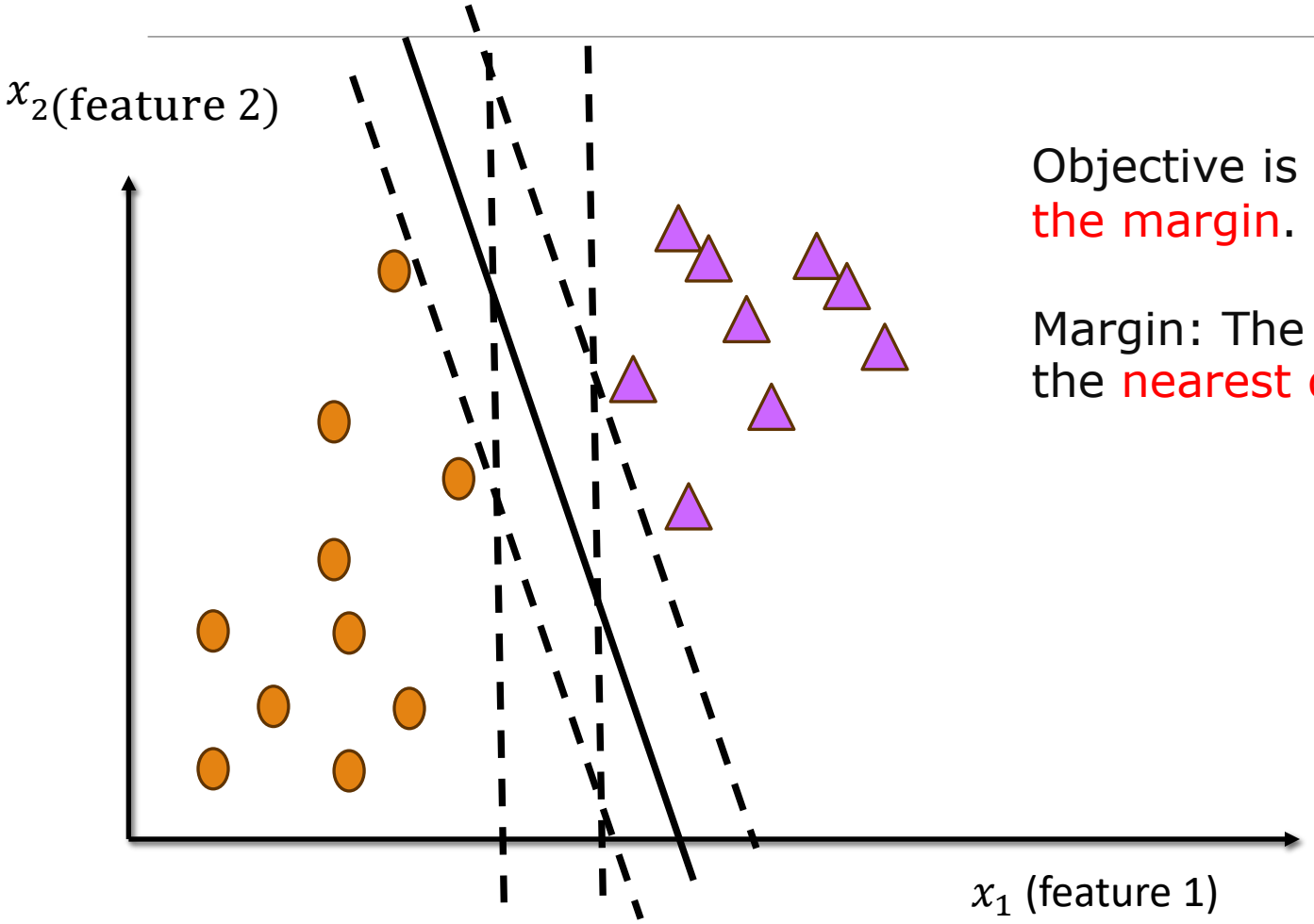
$$\boldsymbol{\mu}_1 = \frac{1}{N_1}\sum_{i=1}^{N} c_i \mathbf{x}_i \text{ and } \boldsymbol{\mu}_2 = \frac{1}{N_2}\sum_{i=1}^{N} (1 - c_i)\mathbf{x}_i$$

Here, $c_i = 1$ for c_1 and $c_i = 0$ for c_2.

$$\boldsymbol{\mu}_1 = [1.83\ 3.83]^T \text{ and } \boldsymbol{\mu}_2 = [-9.75\ -10.75]^T.$$

Maximum Margin Classifiers

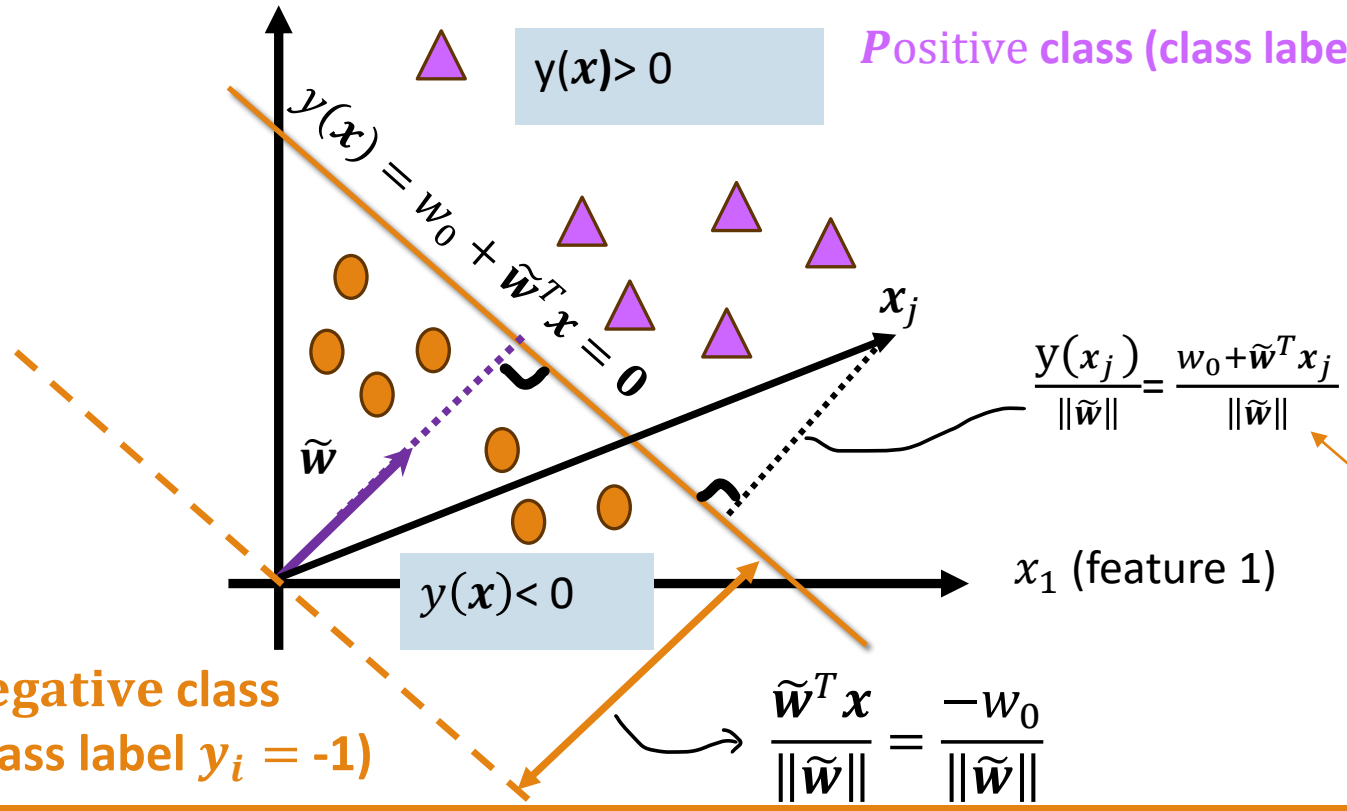
Maximum Margin Classifiers



Objective is to find the hyperplane that maximizes the margin.

Margin: The distance between the hyperplane and the nearest data points from each class

Support Vector Machine (SVM)



$$\boldsymbol{x} = [x_1, x_2, \ldots, x_d]^T$$

$$\tilde{\boldsymbol{w}} = [w_1, w_2, \ldots, w_d]^T$$

(1) $\boldsymbol{x}_i$ positive class $(y_i = 1)$

$$y(\boldsymbol{x}_i) = w_0 + \tilde{\boldsymbol{w}}^T\boldsymbol{x}_i \geq 1^*$$

(2) $\boldsymbol{x}_i$ negative class $(y_i = -1)$

$$y(\boldsymbol{x}_i) = w_0 + \tilde{\boldsymbol{w}}^T\boldsymbol{x}_i \leq -1^*$$

Can we represent (1) and (2) using one equation?

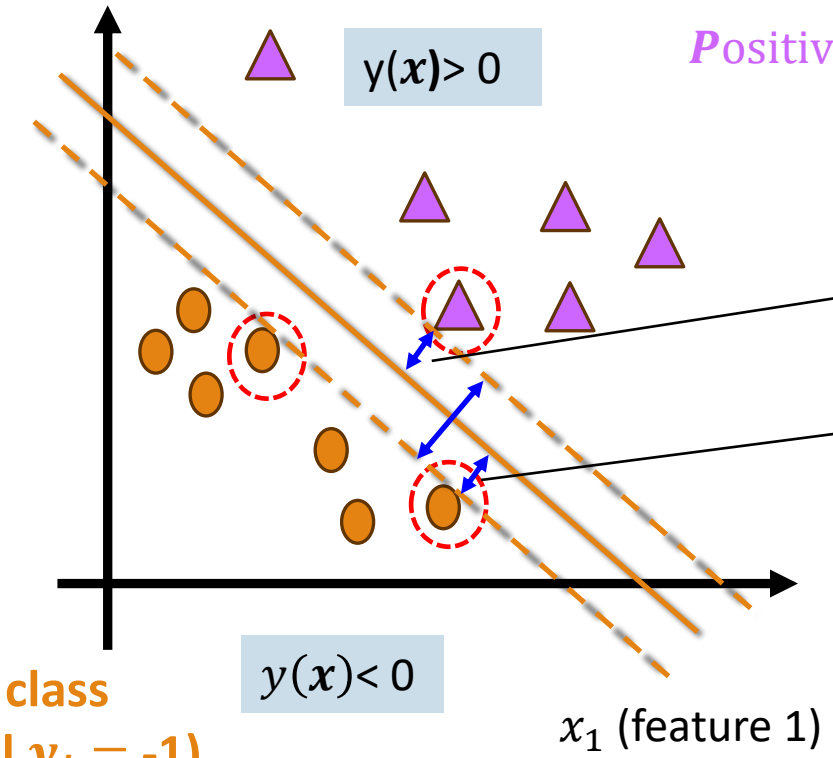
$$y_i\,(w_0 + \tilde{\boldsymbol{w}}^T\boldsymbol{x}_i) \geq 1$$

Distance from a point to a hyperplane

*We replace 0 with -1 and 1, it ensures that the data points on the margin, known as support vectors, are precisely 1 unit away from the decision boundary

Image is adapted from C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

Support Vector Machine (SVM)



$\boldsymbol{x} = [x_1, x_2, \ldots, x_d]^T$

$\tilde{\boldsymbol{w}} = [w_1, w_2, \ldots, w_d]^T$

***P*ositive class (class label $y_i = 1$)**

$$y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,) \geq 1$$

$$p = \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i}{\|\tilde{\boldsymbol{w}}\|}$$

$$n = \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_j}{\|\tilde{\boldsymbol{w}}\|}$$

Optimization problem

$$\arg\max_{\tilde{\boldsymbol{w}}, w_0} \left\{ \frac{1}{\|\tilde{\boldsymbol{w}}\|} \min_i (y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,) \right\}$$

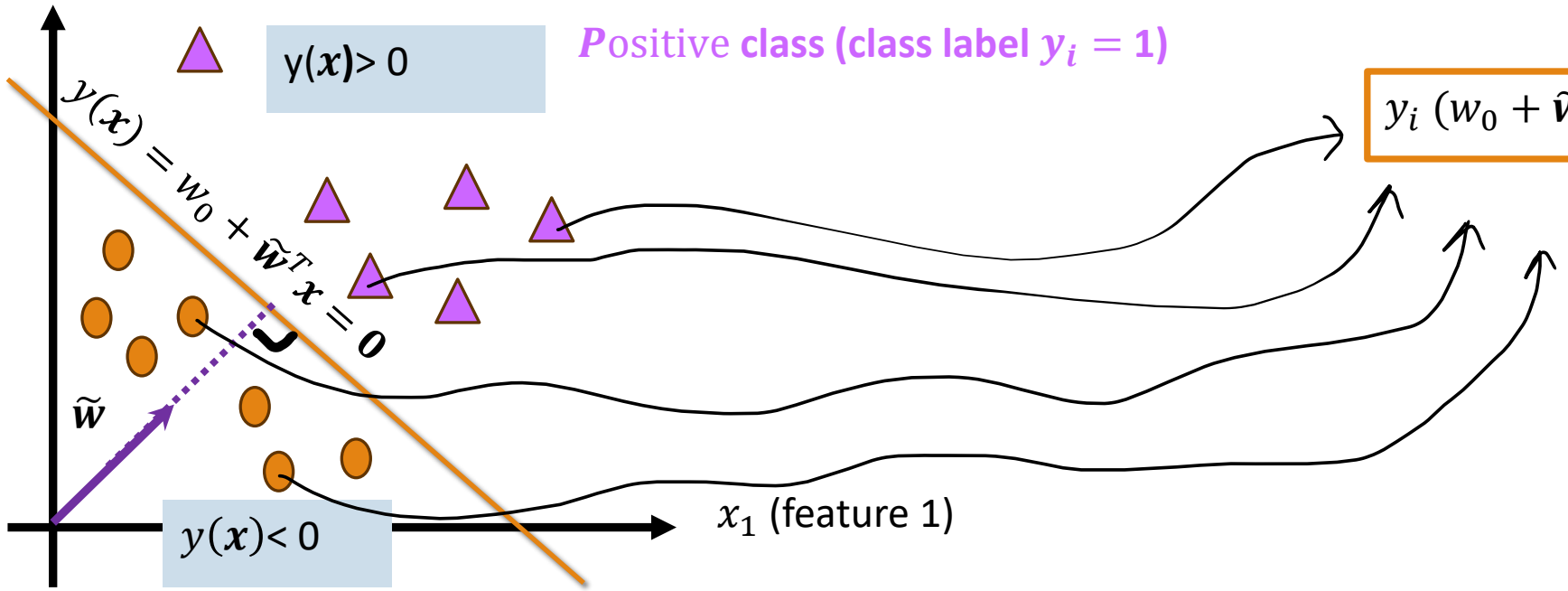
Negative class (class label $y_i = -1$)

Support Vector Machine (SVM)

x_2(feature 2)

$\boldsymbol{x} = [x_1, x_2, \ldots, x_d]^T$

$\tilde{\boldsymbol{w}} = [w_1, w_2, \ldots, w_d]^T$



***P*ositive class (class label $y_i = 1$)**

$$y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,) \geq 1$$

Do we need to consider all data points ?

Negative class (class label $y_i = -1$)

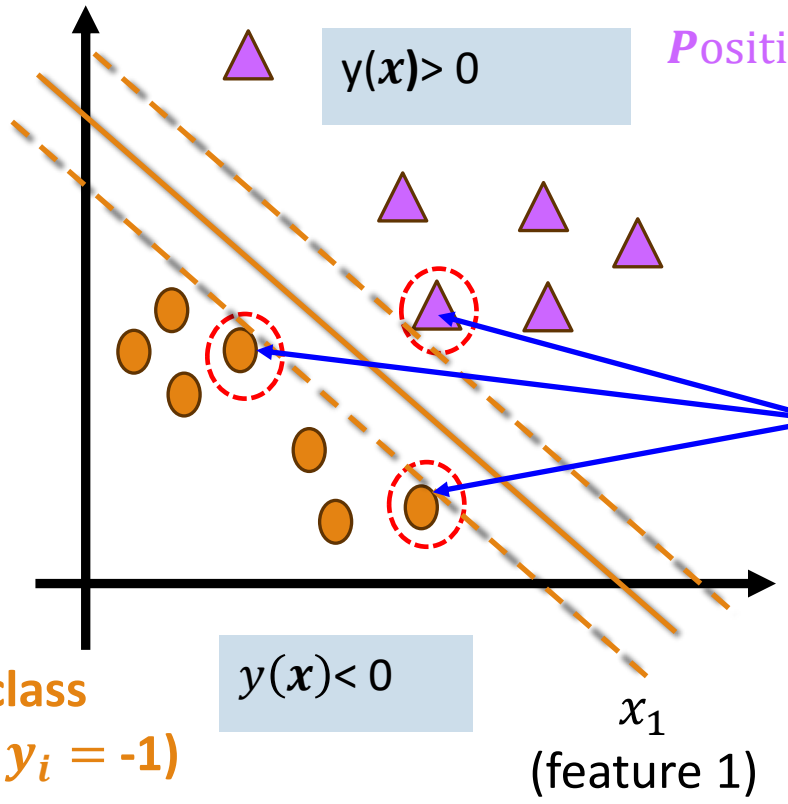
Distance from a point to a hyperplane

*We replace 0 with -1 and 1, it ensures that the data points on the margin, known as support vectors, are precisely 1 unit away from the decision boundary

Image is adapted from C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

Support Vector Machine (SVM)

x_2(feature 2)



$\boldsymbol{x} = [x_1, x_2, \ldots, x_d]^T$

$\tilde{\boldsymbol{w}} = [w_1, w_2, \ldots, w_d]^T$

Positive class (class label $y_i = 1$)

$y_i\ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i) \geq 1$

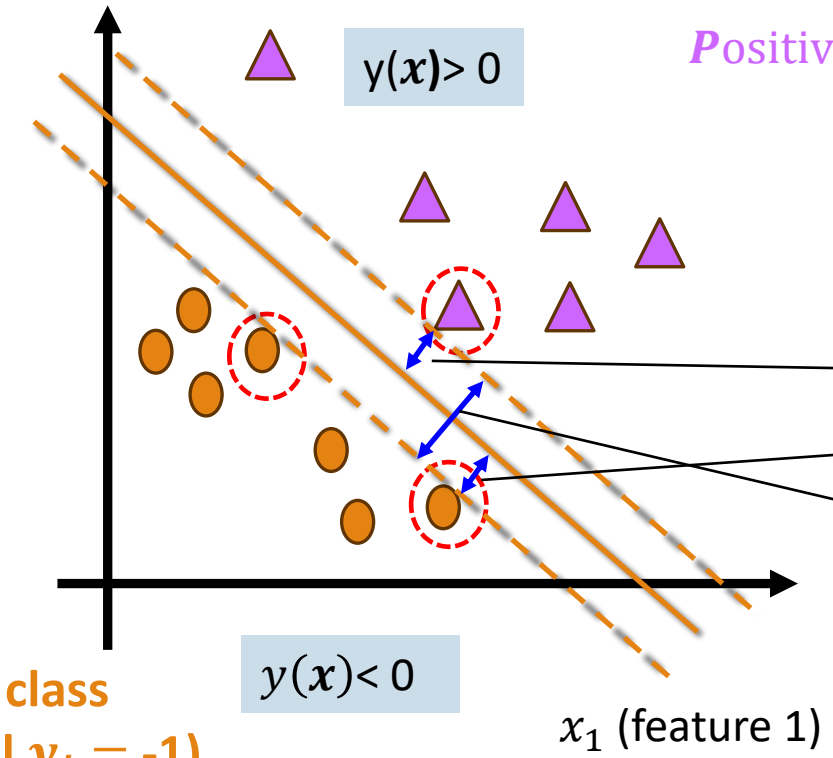
Do we need to consider all data points ?

Support vectors

For support vectors $y_i\ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i) = 1$
or
$\boldsymbol{x}_i$ positive class $(y_i = 1)$: $w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i = 1$
$\boldsymbol{x}_i$ negative class $(y_i = -1)$: $w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i = -1$

Negative class
(class label $y_i = -1$)

Support Vector Machine (SVM)



$$\boldsymbol{x} = [x_1, x_2, \ldots, x_d]^T$$

$$\tilde{\boldsymbol{w}} = [w_1, w_2, \ldots, w_d]^T$$

Positive **class (class label $y_i = 1$)**

$$y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,) \geq 1$$

For support vectors $y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,) = 1$

$$p = \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i}{\|\tilde{\boldsymbol{w}}\|} \qquad n = \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_j}{\|\tilde{\boldsymbol{w}}\|}$$

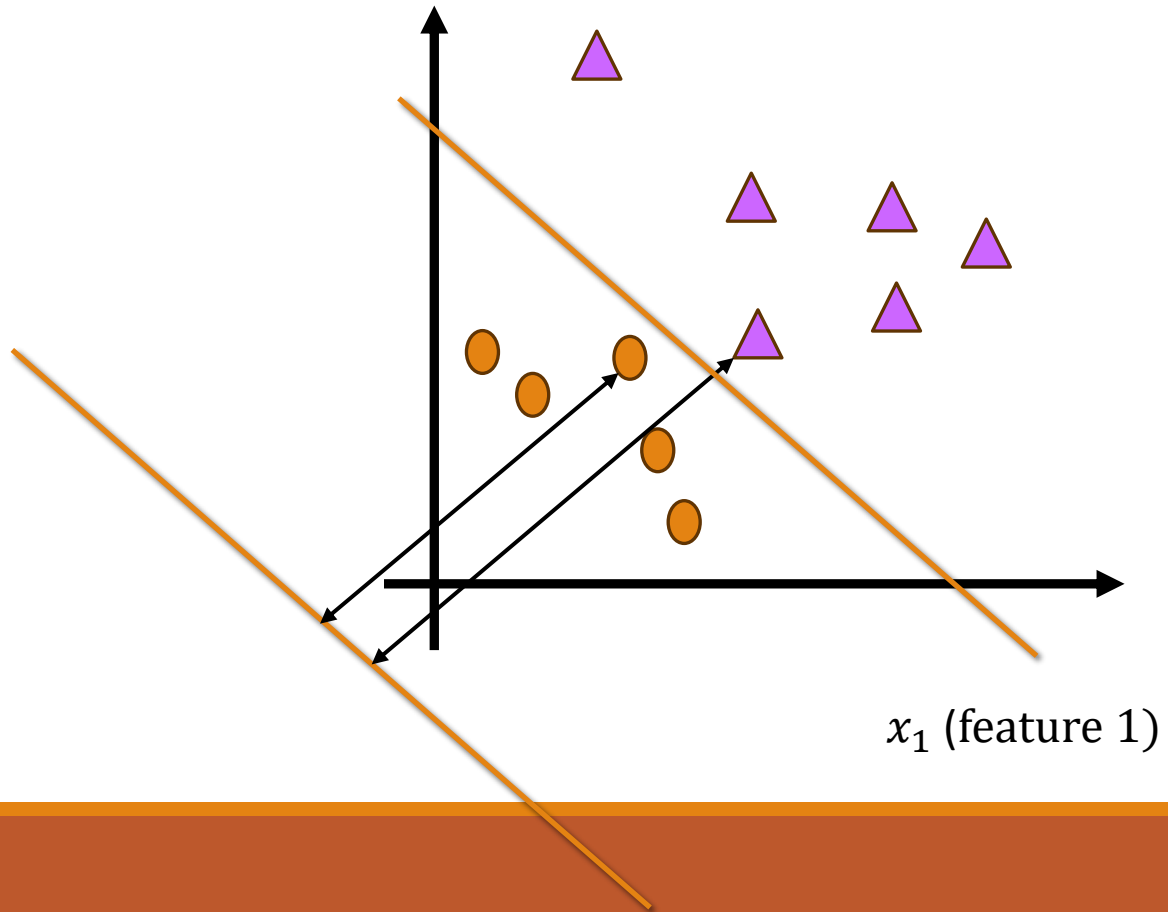
$$p - n = \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i}{\|\tilde{\boldsymbol{w}}\|} - \frac{w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_j}{\|\tilde{\boldsymbol{w}}\|}$$

$$\text{Margin} = \frac{1}{\|\tilde{\boldsymbol{w}}\|} - \frac{(-1)}{\|\tilde{\boldsymbol{w}}\|} = \frac{2}{\|\tilde{\boldsymbol{w}}\|}$$

Negative class (class label $y_i = -1$)

Support Vector Machine (SVM)

- For support vectors, margin is given by $\frac{2}{\|\tilde{\boldsymbol{w}}\|}$
- We need to maximize $\frac{2}{\|\tilde{\boldsymbol{w}}\|}$
- Minimize $\|\tilde{\boldsymbol{w}}\|$
- Only Minimizing $\|\tilde{\boldsymbol{w}}\|$ is sufficient ?
- Need to satisfy $y_i\ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\ ) \geq 1$
- This is required for correct classification of data points



Support Vector Machine (SVM)

- For support vectors, margin is given by $\frac{2}{\|\tilde{\boldsymbol{w}}\|}$
- Minimize $\|\tilde{\boldsymbol{w}}\|$
- Need to satisfy $y_i\ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\ ) \geq 1$
- Optimization problem* (Quadratic)

objective function

$$\underset{\tilde{\boldsymbol{w}}, w_0}{\text{minimize}} \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2$$

$$\text{subject to} \quad y_i\ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\ ) \geq 1$$

Constrains

- Objective function is Quadratic
- Constrains are linear

We can efficiently solve this problem using a QCQP (Quadratically Constrained Quadratic Program) solver

*We utilize the property that the function $f(z) = z^2$ is monotonically increasing for $z \geq 0$

Stanford Engineering | EE364A - Convex Optimization I | Lecture 6

Quadratically constrained quadratic program - Wikipedia

Support Vector Machine (SVM)

Optimization problem

$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}}\ \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2$$
$$\text{subject to}\quad y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)\ \geq 1$$

For support vectors $y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)=1$

Lagrange Function

$$\mathrm{L}(\tilde{\boldsymbol{w}},w_0,\alpha_i)=\frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2-\sum_i\alpha_i\,(y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)-1)$$

Solution

$$\tilde{\boldsymbol{w}}=\sum_i\alpha_i y_i\boldsymbol{x}_i$$

$$\alpha_i=\begin{cases}1, & \text{for support vectors}\\ 0, & \text{else}\end{cases}$$

w_0 ?

For support vectors $y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)=1$

We could use **one support verctor** to find w_0

or take average over **all support vectors**

$$w_0=\frac{1}{N_s}\sum_{j\in S}\left(y_j-\sum_{i\in S}\alpha_i y_i<\boldsymbol{x}_j,\boldsymbol{x}_i>\right)$$

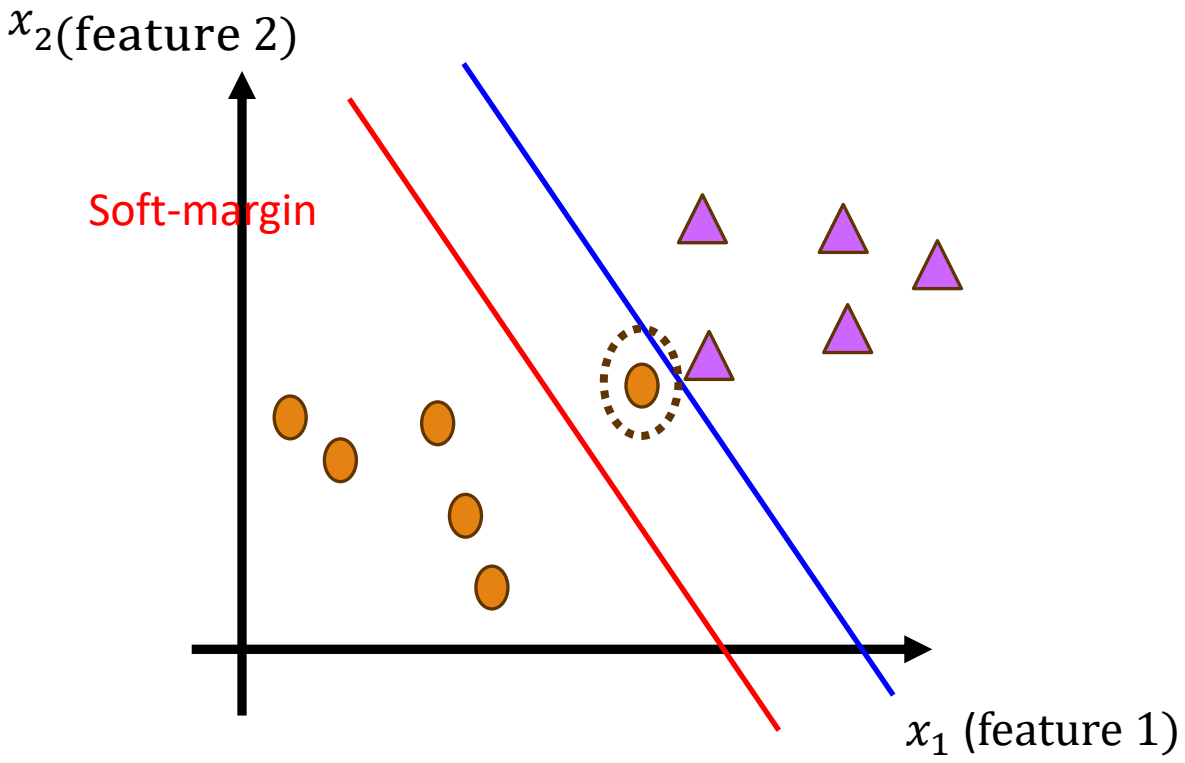
Inner product <,>

Support Vector Machine (SVM)

- $\tilde{\boldsymbol{w}} = \sum_i \alpha_i y_i \boldsymbol{x}_i$ and for support vectors $w_0 = y_i - \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i$
- For a test data sample $\boldsymbol{x}_j$
- $y_j = \tilde{\boldsymbol{w}}^T \boldsymbol{x}_j + w_0 = \sum_i \alpha_i y_i \boldsymbol{x}_i \cdot \boldsymbol{x}_j + w_0$

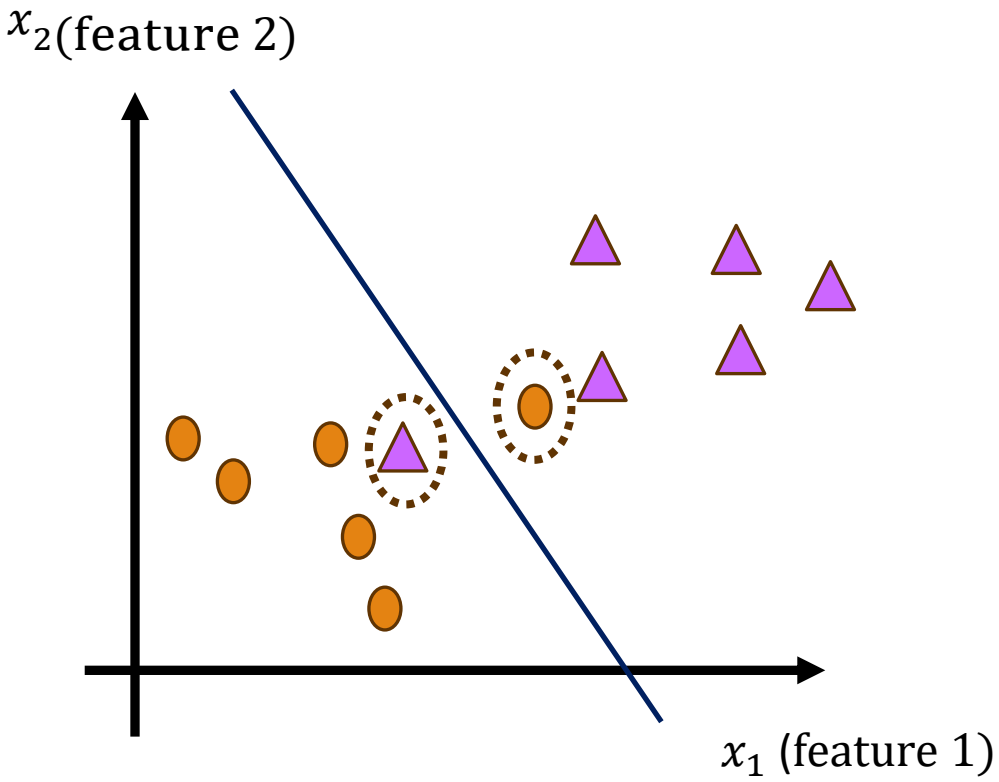
Inner product (dot product) between test sample and support vectors

Support Vector Machine (SVM)



It better to violate some constrains to achieve higher margin

Support Vector Machine (SVM)



It better to violate some constrains to achieve higher margin

Support Vector Machine (SVM)

For separable data

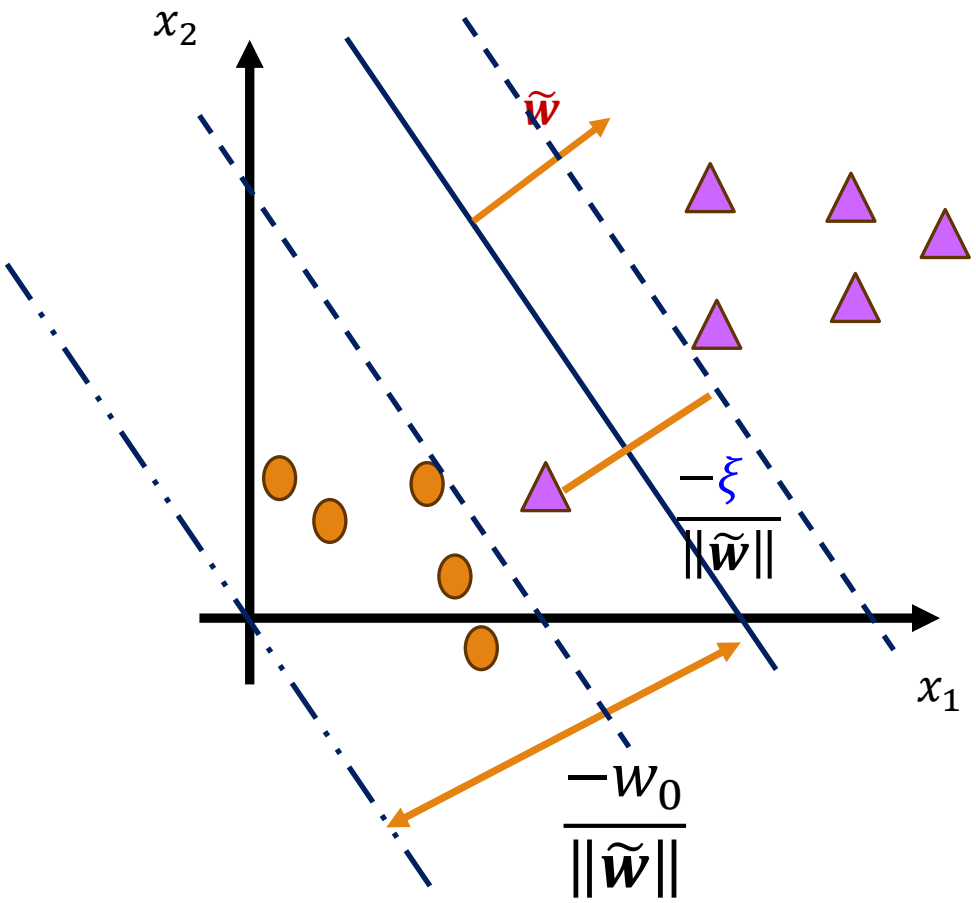
$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}}\ \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2$$
$$\text{subject to}\quad y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)\ \geq 1$$

For linearly non-separable data

$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}}\ \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C\sum_{i=1}^{N}\xi_i$$
$$\text{subject to}\quad y_i\,(w_0+\tilde{\boldsymbol{w}}^T\boldsymbol{x}_i)\ \geq 1-\xi_i$$

C: constant, ξ_i : Positive *slack variable*

Support Vector Machine (SVM)



When margin < 1

$$y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,)\;\geq 1 - \;\xi_i$$

Support Vector Machine (SVM)

$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}}\ \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C\sum_{i=1}^{N}\xi_i$$
$$\text{subject to} \quad y_i\,(w_0 + \tilde{\boldsymbol{w}}^T\boldsymbol{x}_i\,) + \xi_i - 1 \ \geq 0$$

For linearly non-separable data

When margin ≥ 1, we set ξ_i =0

$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}}\ \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C\sum_{i=1}^{N}\max(0, 1 - y_i\,(w_0 + \tilde{\boldsymbol{w}}^T\boldsymbol{x}_i\,)$$

Support Vector Machine (SVM)

$$\underset{\tilde{w},w_0}{\text{minimize}}\,\frac{1}{2}\|\tilde{w}\|^2 + C\sum_{i=1}^{N}\max(0, 1 - y_i\,(w_0 + \tilde{w}^T x_i))$$

Maximize the margin

Minimize the classification error (misclassification)

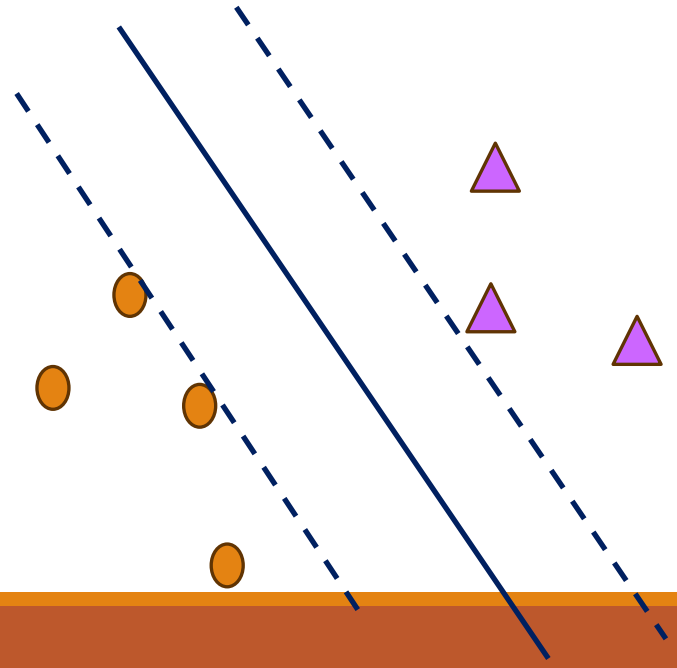
Higher $\left[C\ \sum_{I=1}^{N}\max\left(0, 1 - \left(w_0 + \tilde{w}^T x_i\right)\right)\right]$ means more misclassification

Support Vector Machine (SVM)

$$\underset{\widetilde{\boldsymbol{w}},w_0}{\text{minimize}} \frac{1}{2}\|\widetilde{\boldsymbol{w}}\|^2 \quad + C \sum_{i=1}^{N} \max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$$

Hinge loss

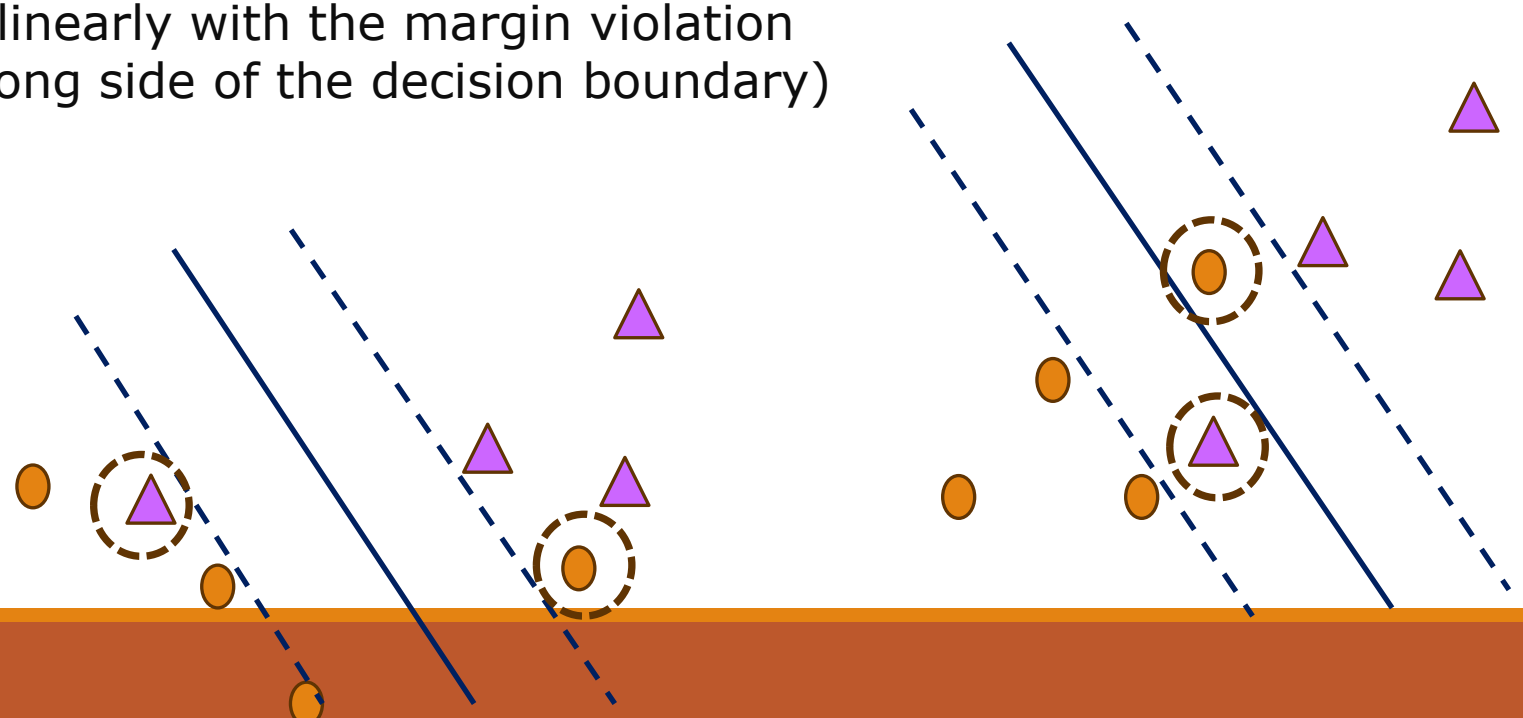
- $\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$
- The hinge loss is zero for correctly predicted class label with a margin of at least 1



Support Vector Machine (SVM)

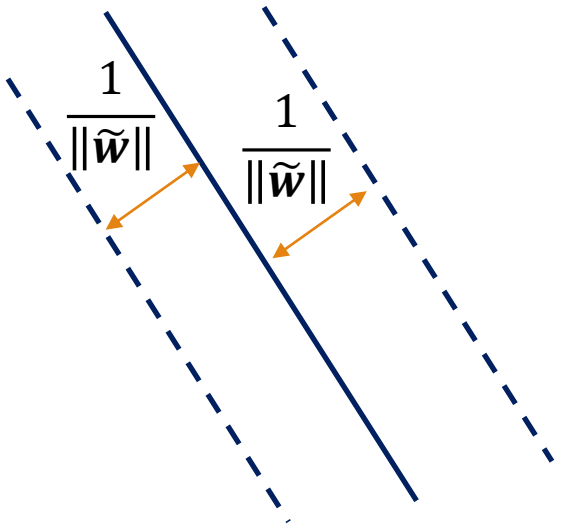
$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}} \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C \sum_{i=1}^{N} \max(0, 1 - y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$$

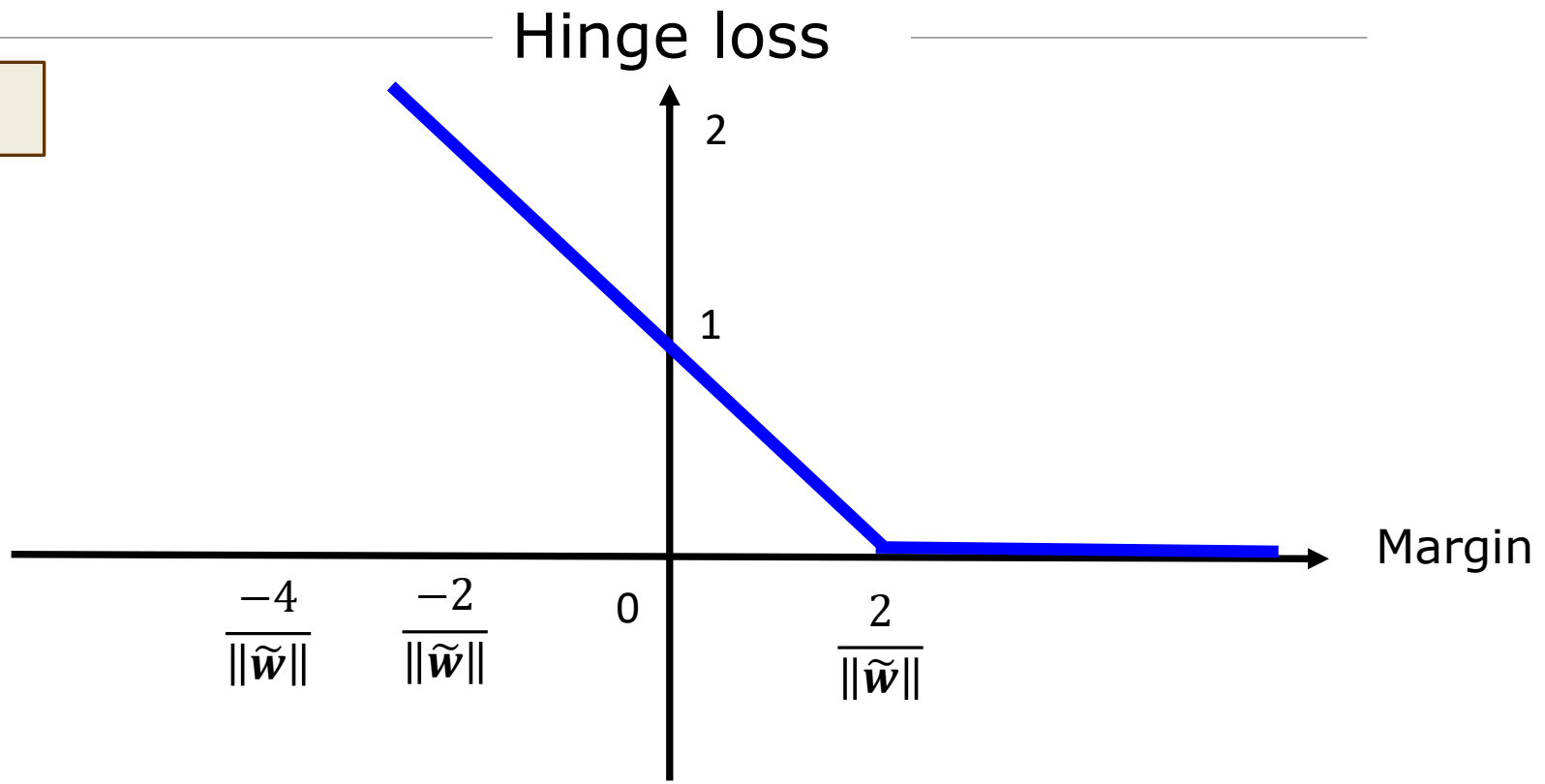
- Hinge loss increases linearly with the margin violation (within the margin or wrong side of the decision boundary)



Support Vector Machine (SVM)

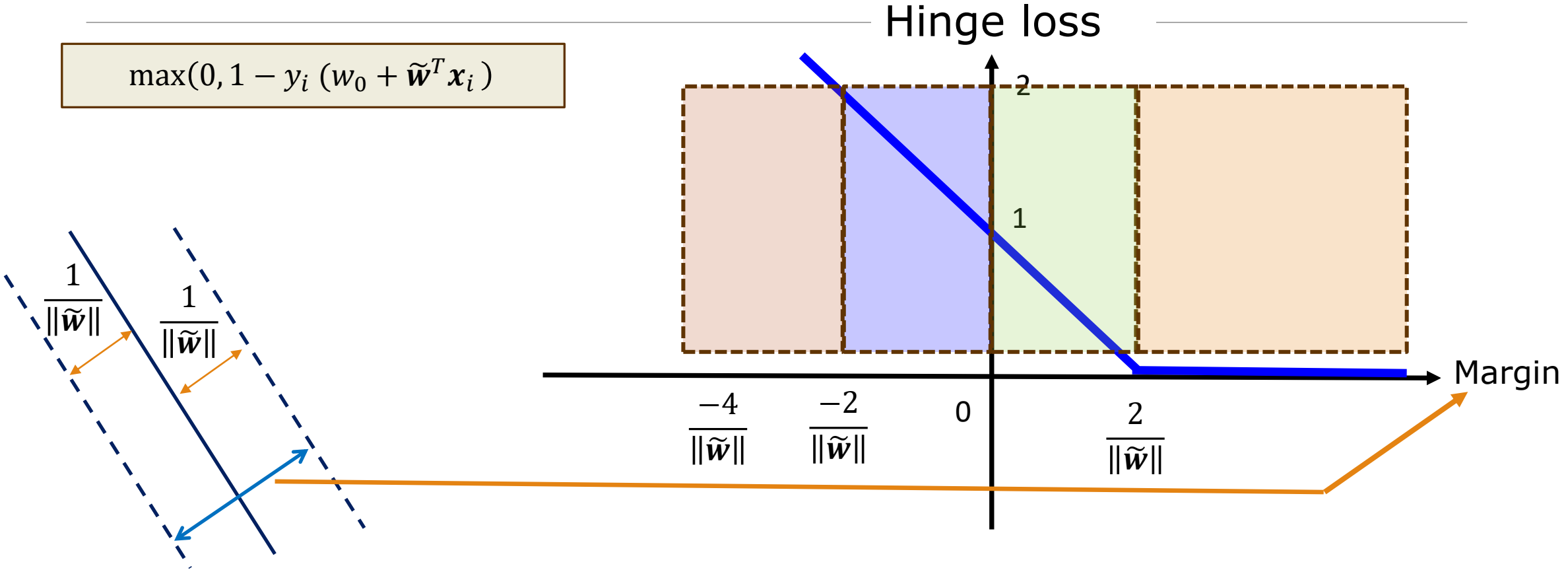
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,))$$





Support Vector Machine (SVM)

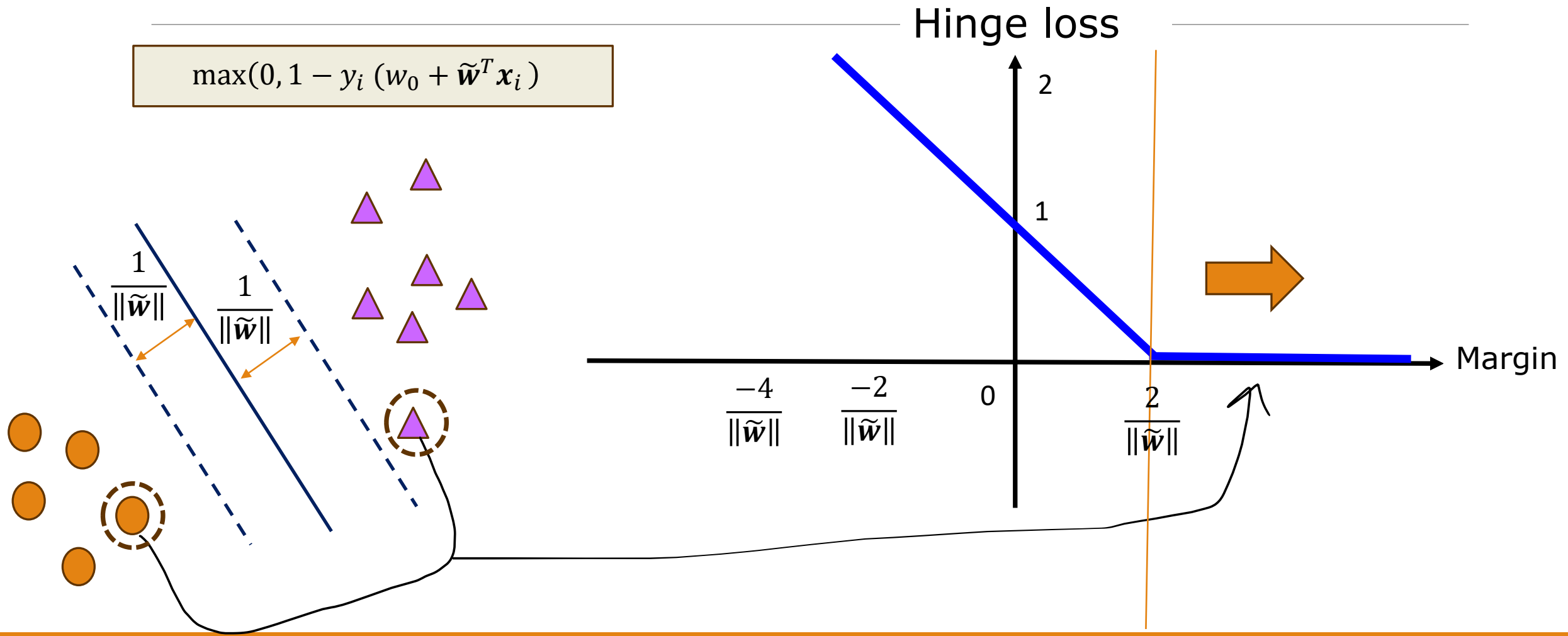
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,))$$



Can you identify regions ?

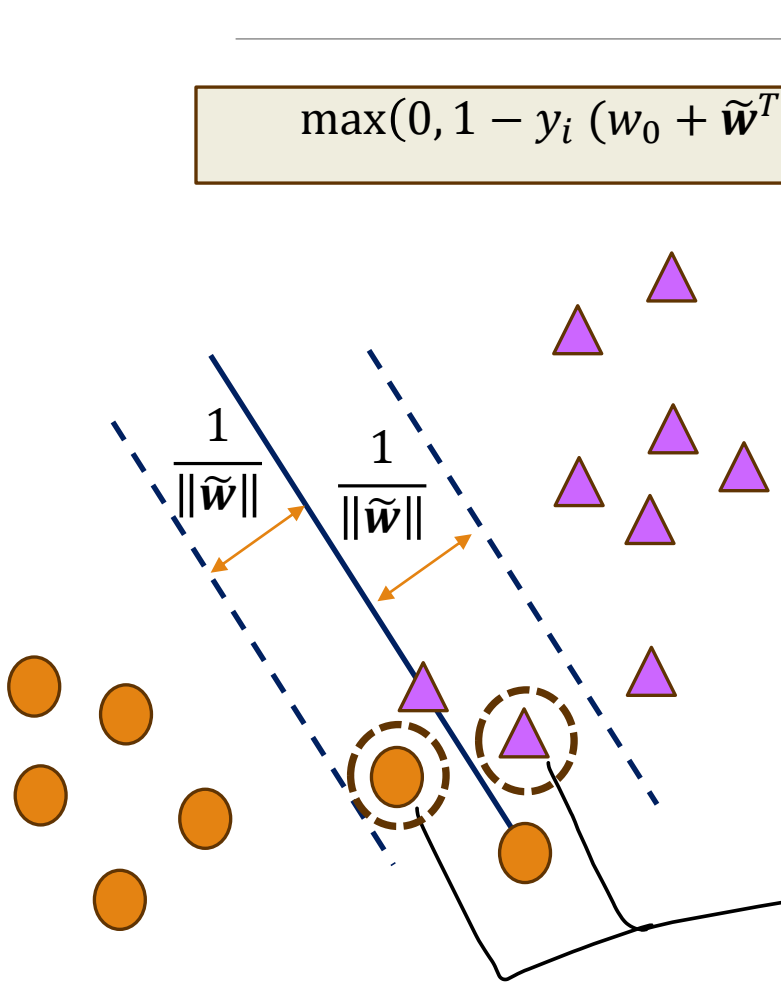
Support Vector Machine (SVM)

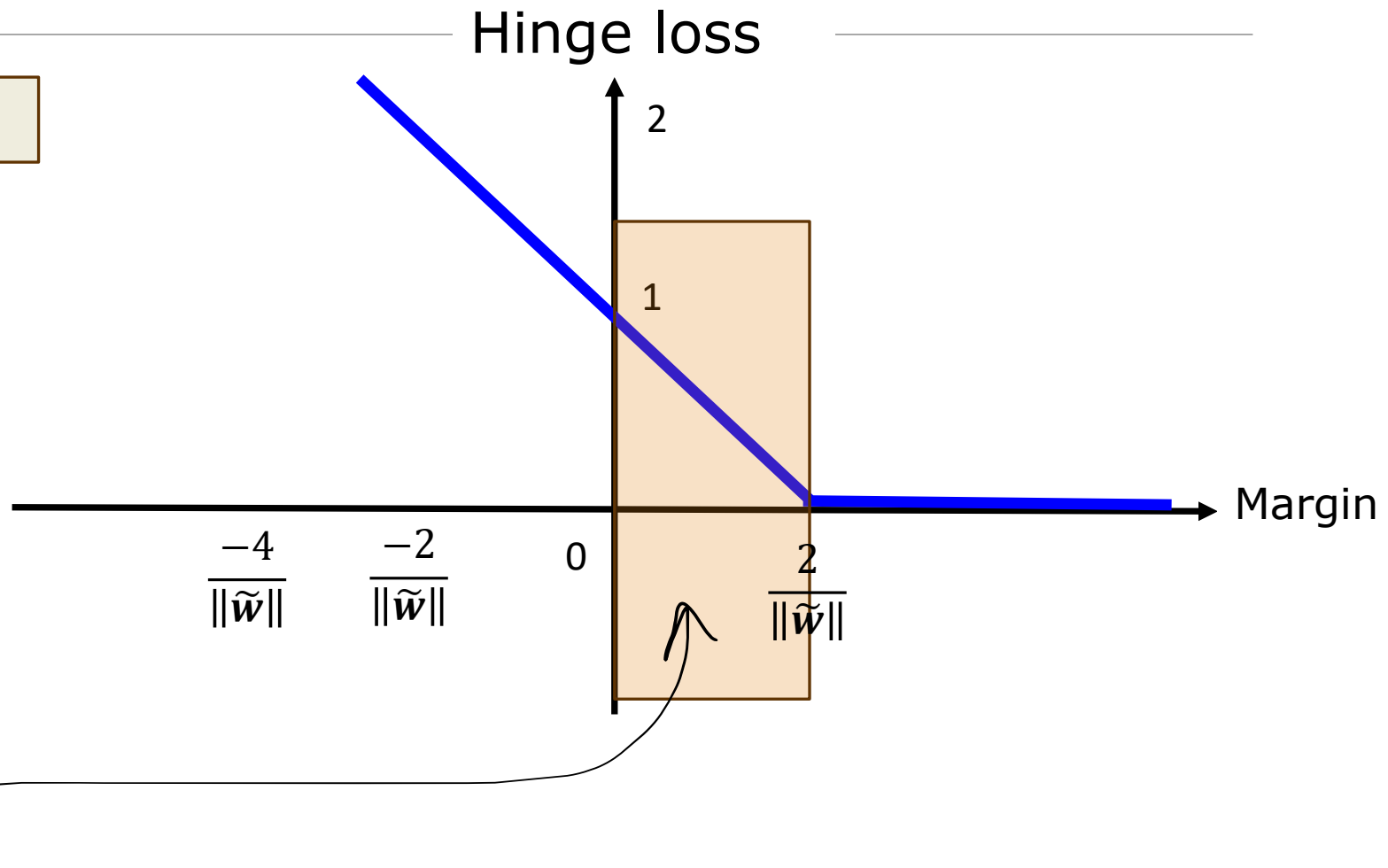
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,))$$



Support Vector Machine (SVM)

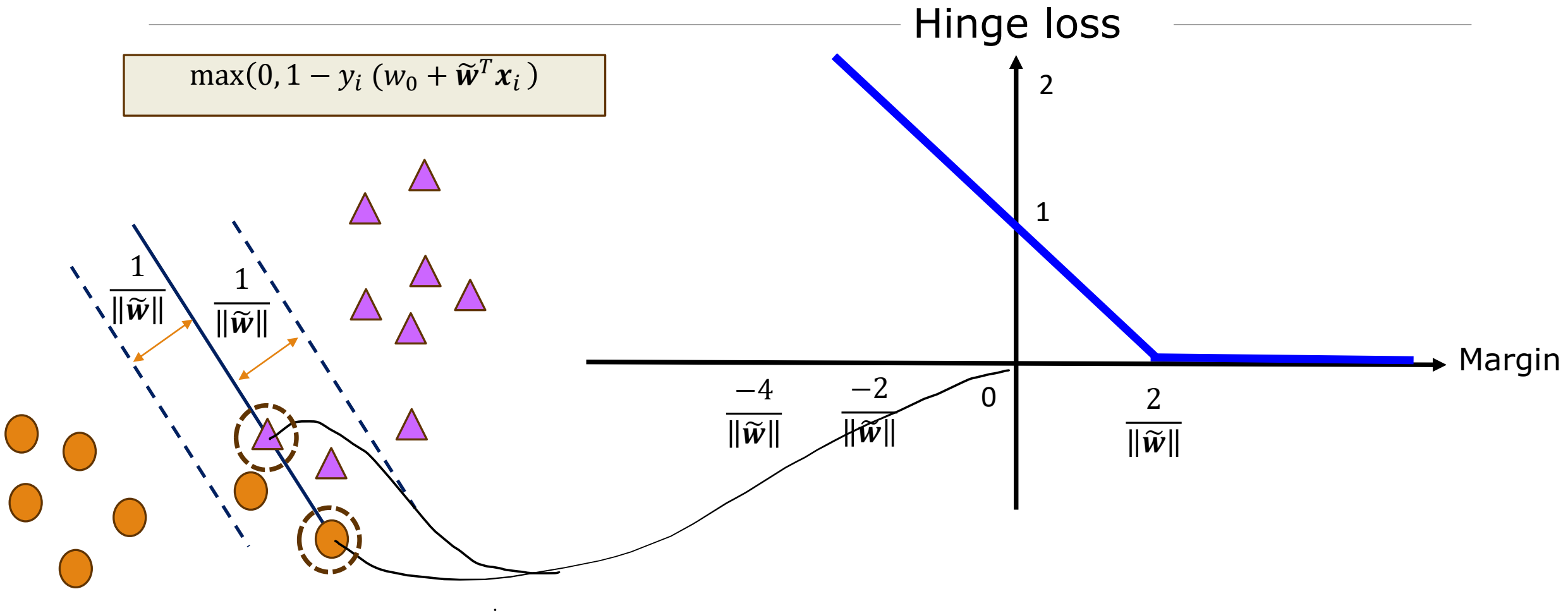
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,)$$





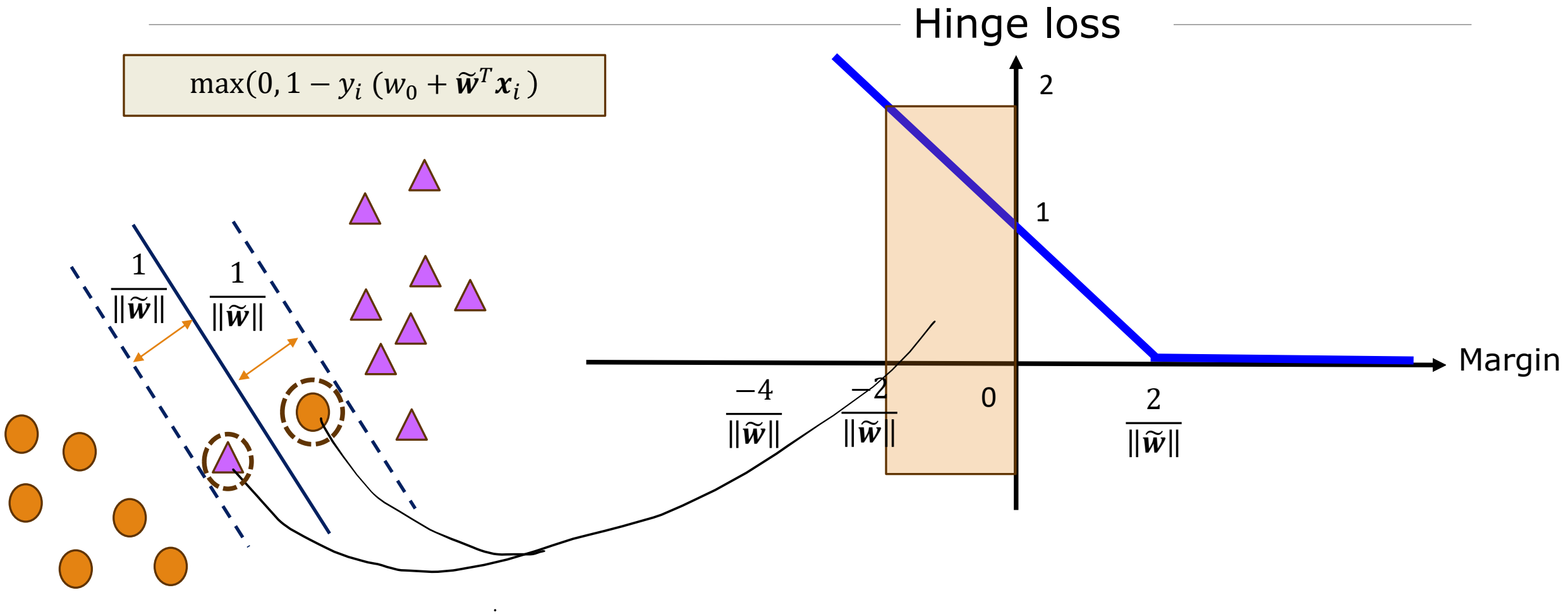
Support Vector Machine (SVM)

$$\max(0, 1 - y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$$



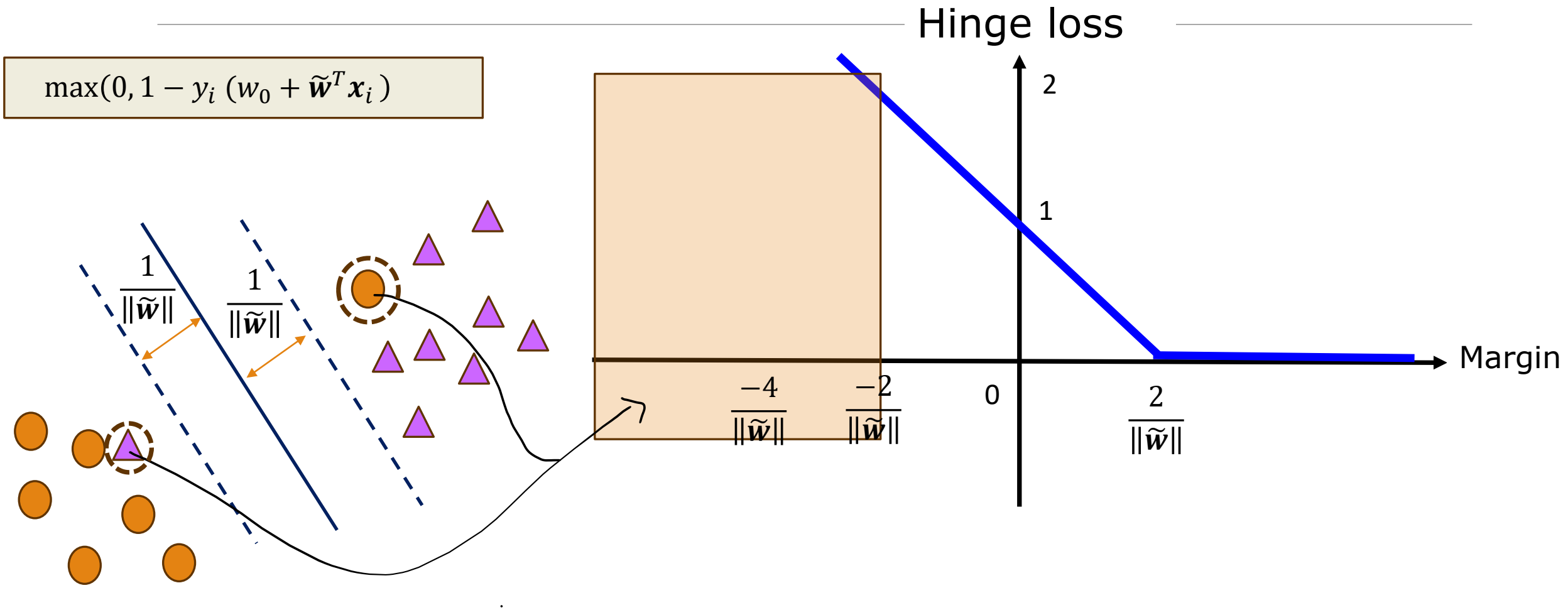
Support Vector Machine (SVM)

$$\max(0, 1 - y_i\ (w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\ )$$



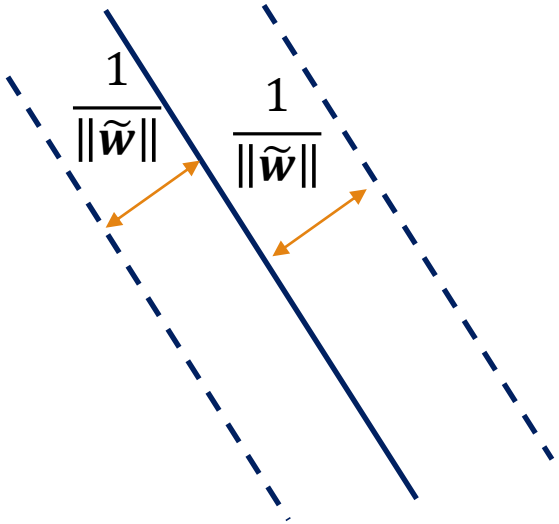
Support Vector Machine (SVM)

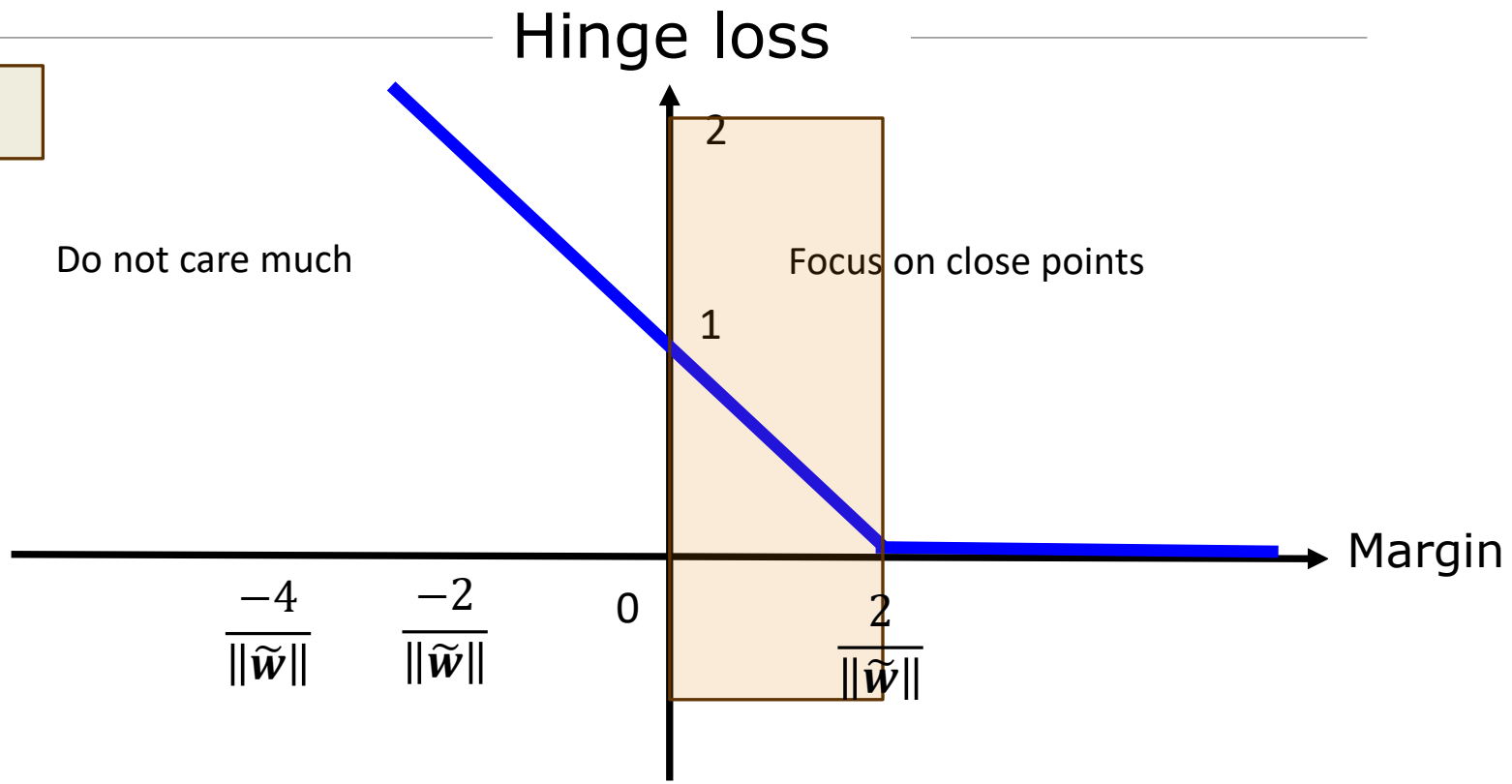
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,)$$



Support Vector Machine (SVM)

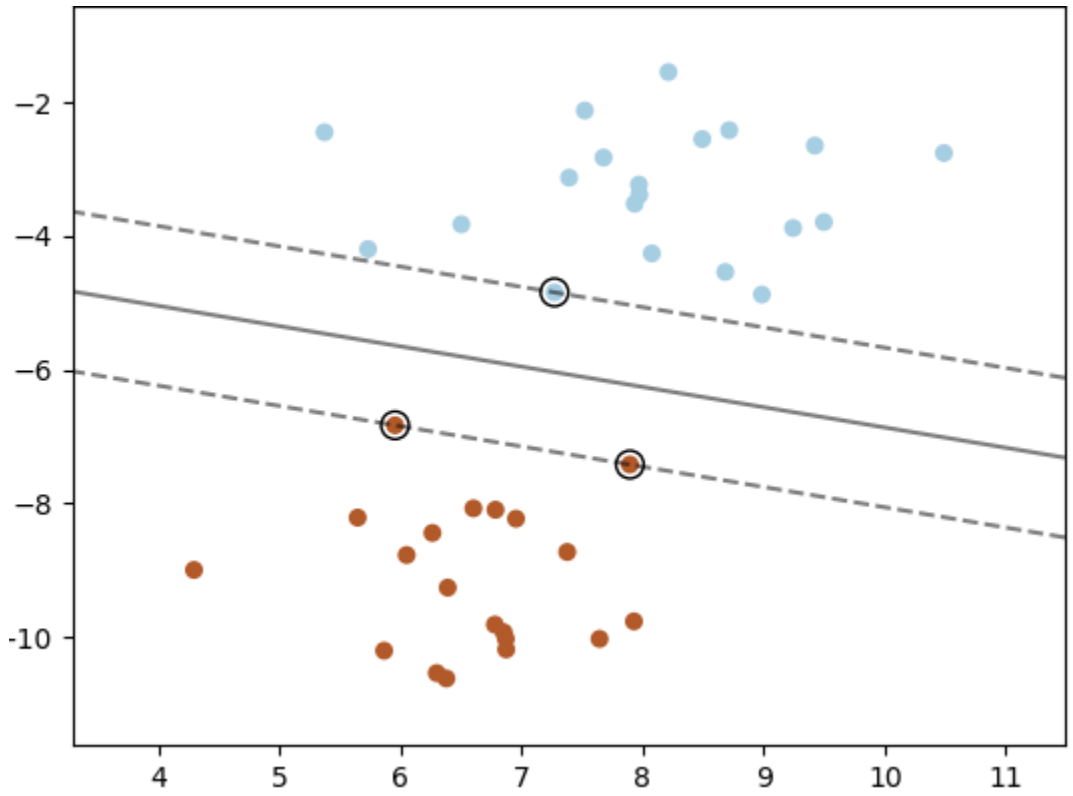
$$\max(0, 1 - y_i\,(w_0 + \widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$$





SVM

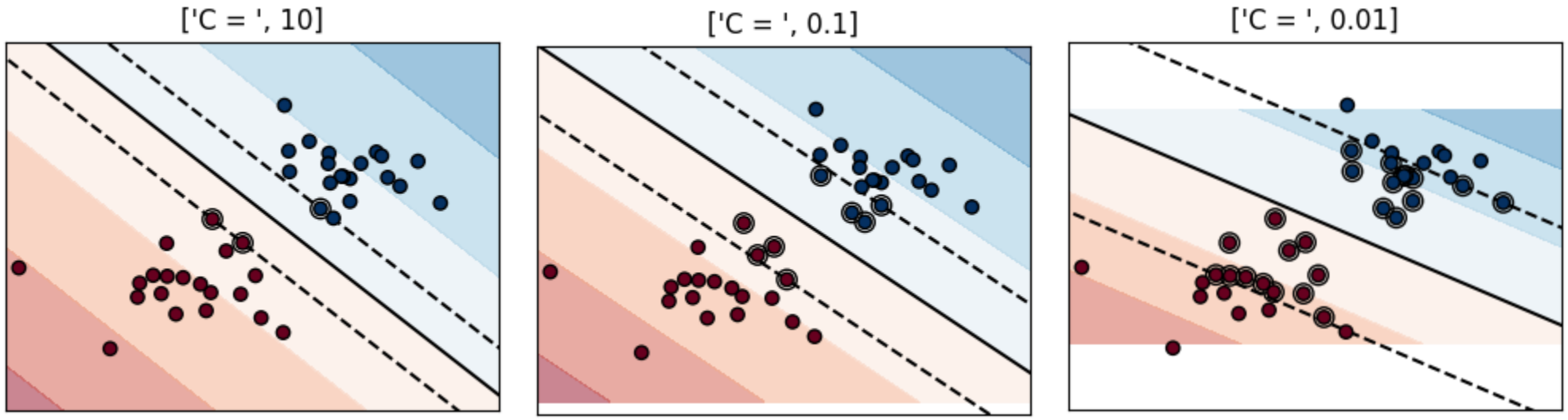
SVM: Maximum margin separating hyperplane — scikit-learn 1.4.1 documentation



SVM

https://scikit-learn.org/stable/auto_examples/svm/plot_svm_margin.html#sphx-glr-auto-examples-svm-plot-svm-margin-py

$$\underset{\tilde{\boldsymbol{w}},w_0}{\text{minimize}} \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C\sum_{I=1}^{N} \max(0, 1 - y_i\,(w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i\,)$$



A large value of C basically tells our model that we do not have that much faith in our data's distribution and will only consider points close to line of separation.

"A small value of C includes more/all the observations, allowing the margins to be calculated using all the data in the area." SVM Margins Example — scikit-learn 1.3.0 documentation

In class activity

$$L(\tilde{w}, w_0, \alpha_i) = \frac{1}{2}\|\tilde{w}\|^2 - \sum_i \alpha_i \left(y_i \left(w_0 + \tilde{w}^T x_i\right) - 1\right)$$

How to identify support vectors ?

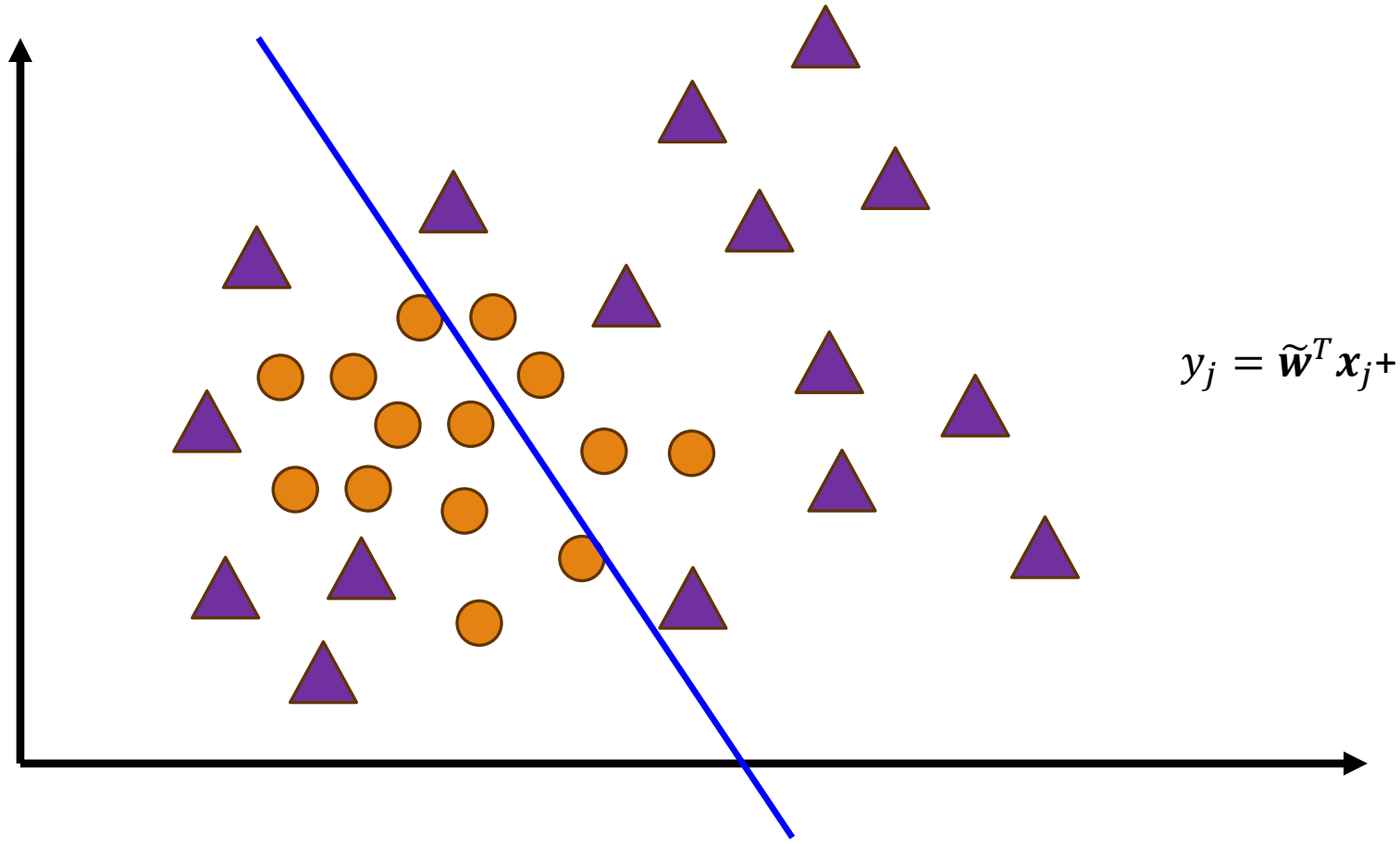
$$\underset{\tilde{w}, w_0}{\text{minimize}} \frac{1}{2}\|\tilde{w}\|^2 + C \sum_{I=1}^{N} \max(0, 1 - y_i \left(w_0 + \tilde{w}^T x_i\right)$$

For mis-classifications $\left(-y_i \left(w_0 + \tilde{w}^T x_i\right)\right) > 0$



See: https://scikit-learn.org/stable/modules/svm.html#svm-mathematical-formulation

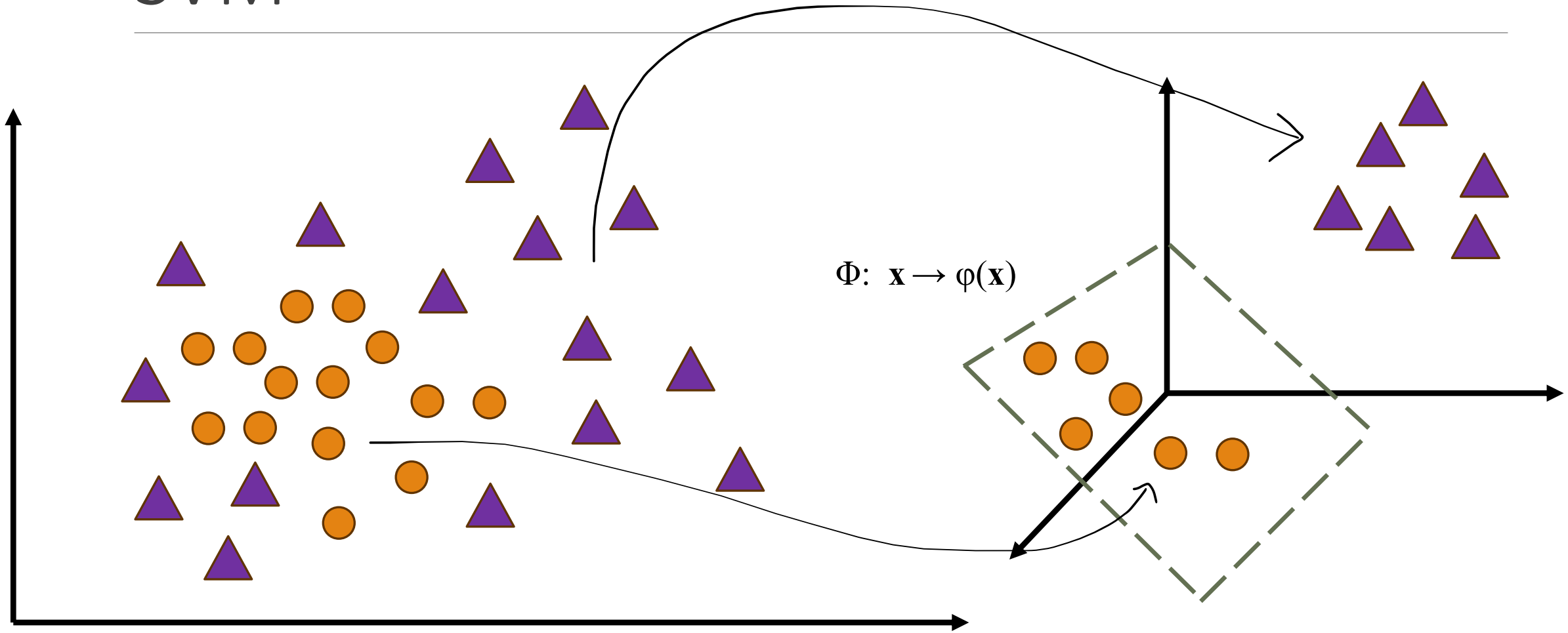
SVM



How to address this ?

$$y_j = \tilde{\boldsymbol{w}}^T \boldsymbol{x}_j + w_0 = \sum_i \alpha_i y_i \boldsymbol{x}_i \bullet \boldsymbol{x}_j + w_0$$

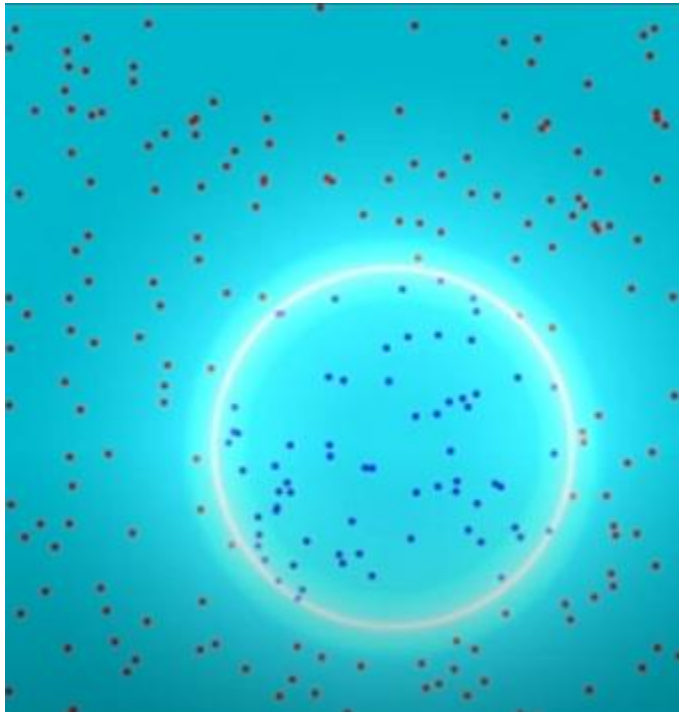
SVM

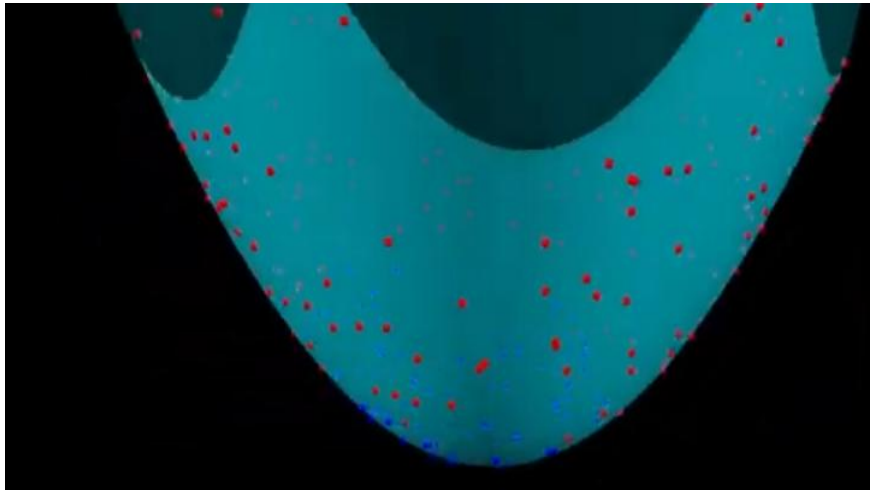


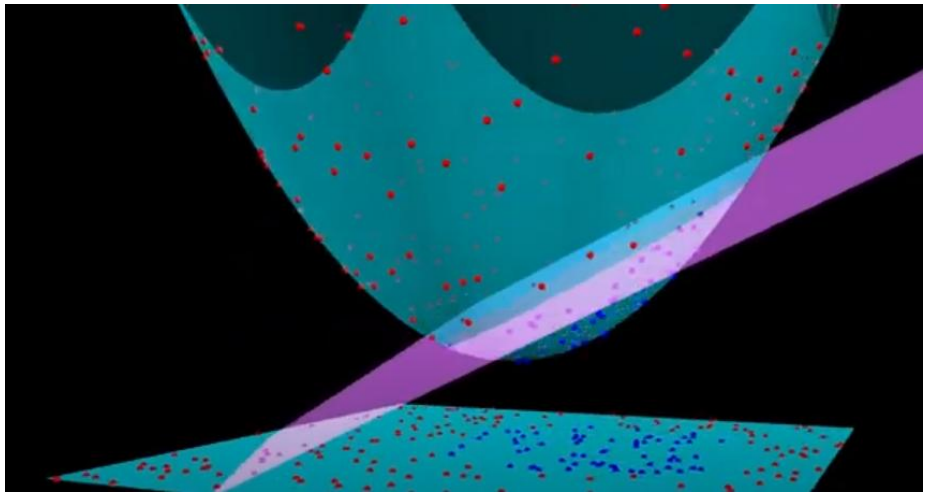
Kernel trick

SVM with polynomial kernel visualization – YouTube

$k(x,y) = [x\ y, (x^2+y^2)]$







visualization from www.udiprod.com

Kernel trick for SVM

$K(\mathbf{x}, \mathbf{y}) = \boldsymbol{\varphi}(\mathbf{x}) \cdot \boldsymbol{\varphi}(\mathbf{y})$

$y_j = \sum_i \alpha_i y_i \varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_j) + w_0$

$y_j = \sum_i \alpha_i y_i \, \mathrm{K}(\boldsymbol{x}_i , \boldsymbol{x}_j) + w_0$

This results in a nonlinear decision boundary within the original feature space

Kernels

- Linear kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)$$

$K(\boldsymbol{x}, \boldsymbol{z})$ is larger ➔ $\boldsymbol{x}$ and $\boldsymbol{z}$ are similar

- Polynomial kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)^{\boldsymbol{d}}$$

- Gaussian kernel (radial basis function kernel, or RBF kernel)

$$K(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(\frac{-\|\boldsymbol{x}-\boldsymbol{z}\|_2^2}{2\sigma^2}\right)$$

$K(\boldsymbol{x}, \boldsymbol{z}) \to 1$, when $\boldsymbol{x}$ and $\boldsymbol{z}$ are close

$K(\boldsymbol{x}, \boldsymbol{z}) \to 0$, when $\boldsymbol{x}$ and $\boldsymbol{z}$ arefar apart

$c \geq 0$

Kernel Matrix for SVM

$y_j = \sum_i \alpha_i y_i \, \mathrm{K}(\boldsymbol{x}_i , \boldsymbol{x}_j) + w_0$

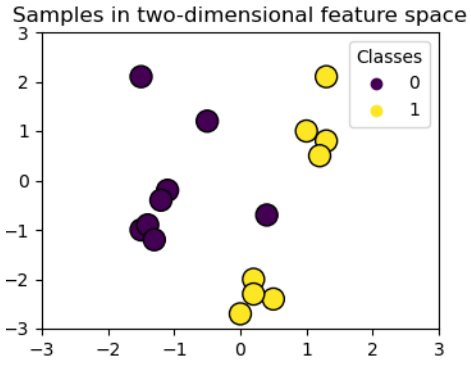
- Only the support vectors contribute to the decision boundary

Gram matrix (Kernel matrix)

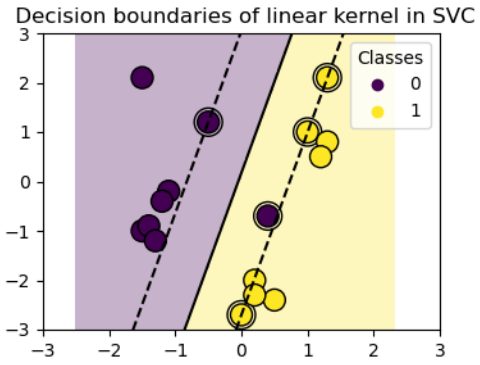
$$\boldsymbol{K} = \begin{bmatrix} \mathrm{K}(\boldsymbol{x}_1 , \boldsymbol{x}_1) & \mathrm{K}(\boldsymbol{x}_1 , \boldsymbol{x}_2) & \cdots & \mathrm{K}(\boldsymbol{x}_1 , \boldsymbol{x}_N) \\ \mathrm{K}(\boldsymbol{x}_2 , \boldsymbol{x}_1) & \mathrm{K}(\boldsymbol{x}_2 , \boldsymbol{x}_2) & & \mathrm{K}(\boldsymbol{x}_2 , \boldsymbol{x}_N) \\ \vdots & & \ddots & \vdots \\ \mathrm{K}(\boldsymbol{x}_N , \boldsymbol{x}_1) & \mathrm{K}(\boldsymbol{x}_N , \boldsymbol{x}_2) & \cdots & \mathrm{K}(\boldsymbol{x}_N , \boldsymbol{x}_N) \end{bmatrix}$$

SVM with Kernel: Example

- Plot classification boundaries with different SVM Kernels



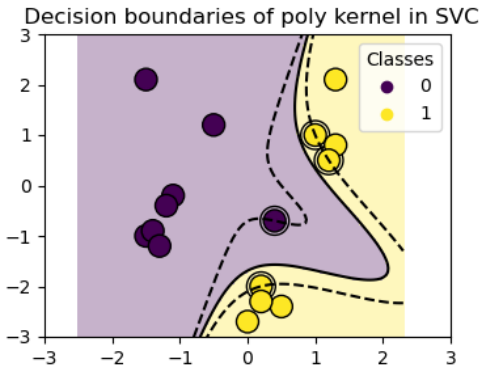
Cannot be separable by a straight line



Linear Kernel

$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z})$

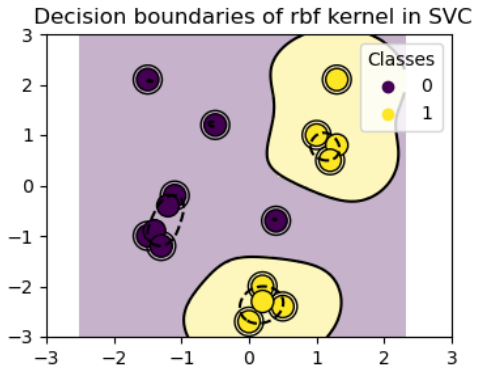
"untransformed feature space and do not able to fully separate classes"[1].



Polynomial kernel

$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)^{\boldsymbol{d}}$

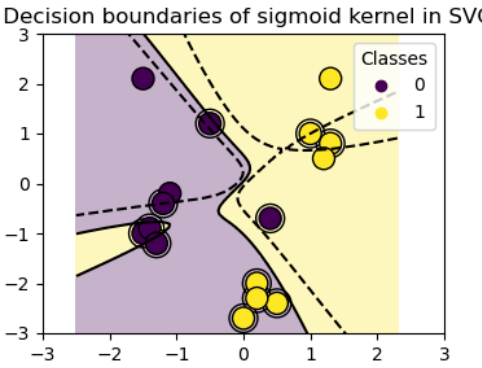
"Adapts well to the training data"[1], able to separate classes.



Radial basis function (RBF)

$K(\boldsymbol{x}, \boldsymbol{z}) = \exp(-\Upsilon \| \boldsymbol{x} - \boldsymbol{z} \|_2^2)$

"Decision boundaries tend to contract around data points that are close to each other "[1].



Sigmoid Kernel

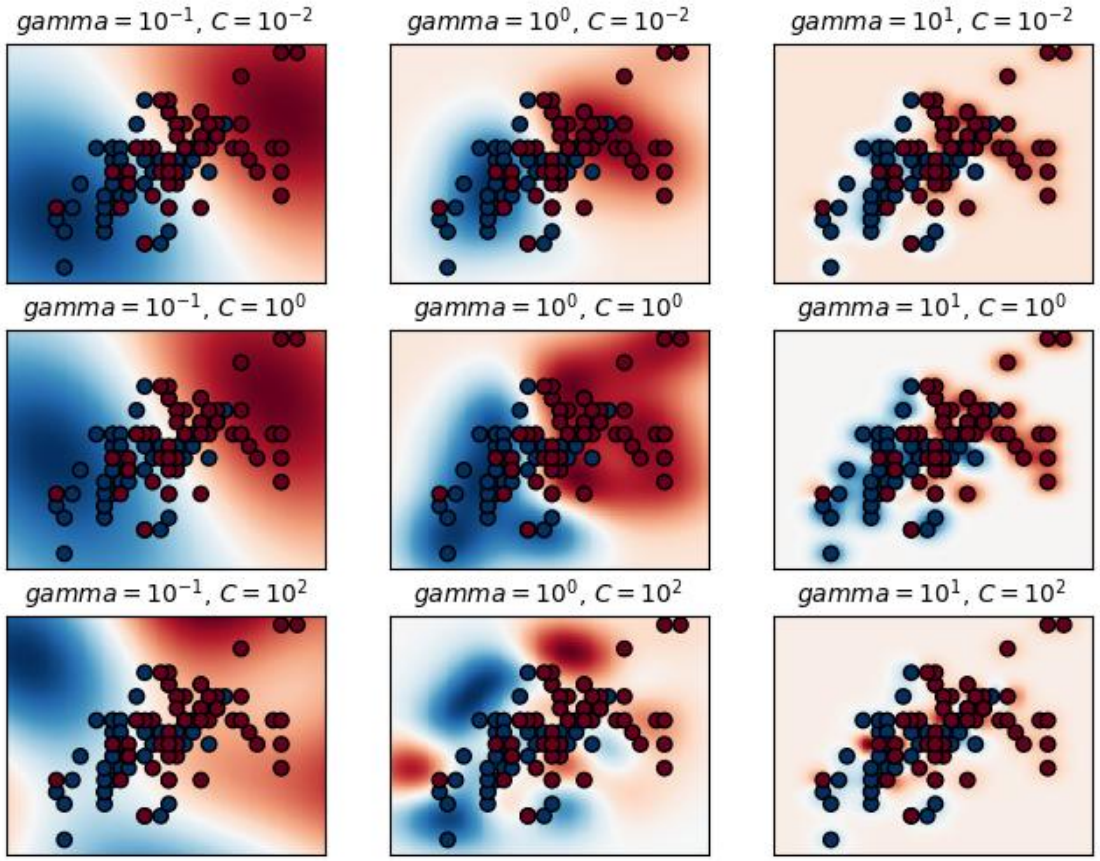
$K(\boldsymbol{x}, \boldsymbol{z}) = \tanh(\Upsilon \boldsymbol{x}^T \boldsymbol{z} + r)$

"Decision boundaries are curved and irregular may not generalize well to unseen data "[1].

Images from [1] https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html

SVM with Kernel: Example

https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html#sphx-glr-auto-examples-svm-plot-rbf-parameters-py



$$K(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(-\gamma \ \| \boldsymbol{x} - \boldsymbol{z} \|_2^2\right)$$

$$\underset{\tilde{\boldsymbol{w}}, w_0}{\text{minimize}} \frac{1}{2}\|\tilde{\boldsymbol{w}}\|^2 \quad + C \sum_{I=1}^{N} \max(0, 1 - y_i \ (w_0 + \tilde{\boldsymbol{w}}^T \boldsymbol{x}_i))$$

- For larger values of C, a smaller margin
- A small value of gamma may cause overfitting, where the decision boundary becomes overly sensitive to individual data points. Conversely, a large value of gamma could lead to underfitting
- With a large gamma in the Gaussian kernel, the distribution becomes broader and smoother. The kernel assigns comparable weights to a wider area surrounding each data point.

SVM

Advantages:

- Effective in high-dimensional spaces
- Can be used even when number of features ≫ the number of samples
- Memory efficient as it uses of a subset of training points (support vectors)
- Non-linear decision boundaries with kernel trick
- Can be extend to regression task

- **Disadvantages**
  - Binary class, need alternative approaches to extend multiclass
  - Probability estimates are not given
  - Appropriate kernel function need to be selected, and regularization term need to be tuned
  - Computation and memory usage can be high for large scale problems

Question

Consider a dataset consisting of three flower classes: Iris Setosa, Iris Versicolor, and Iris Virginica. In this dataset each data sample is represented by a 4-dimensional vector. After the learning process, a linear classifier is given as follows:

$$\boldsymbol{W} = \begin{bmatrix} 0.41 & 1.46 & -2.26 & -1.02 \\ 0.42 & -1.61 & 0.57 & -1.40 \\ -1.70 & -1.53 & 2.47 & 2.55 \end{bmatrix}^T, \text{ and } \boldsymbol{b} = \begin{bmatrix} 0.26 & 1.09 & -1.21 \end{bmatrix}^T.$$

Suppose that two data samples $\boldsymbol{x}_1$ and $\boldsymbol{x}_2 \in \mathbb{R}^{4\times 1}$ are fed to this linear classifier. $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are given by

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_1 & \boldsymbol{x}_2 \end{bmatrix} = \begin{bmatrix} 4 & 3 & 4 & 1 \\ 8 & 1 & 3 & 2 \end{bmatrix}^T \in \mathbb{R}^{4\times 2}.$$

Output of the linear classifier for $i-$th data sample is given by $\boldsymbol{y}_i = f\left(\boldsymbol{W}^T \boldsymbol{x}_i + \boldsymbol{b}\right) \in \mathbb{R}^{3\times 1}$. Here, $f(\cdot)$ is the sigmoid function. Suppose that probability of $i-$th data sample belongs to $j-$th class denoted by $P_{j,i}$. What are the probabilities of both data samples?





Thank You

Q & A